

# 1. INTRODUCTION

In today's digital economy, the rapid growth of credit-based transactions has increased the need for accurate credit risk assessment. With millions relying on credit cards, financial institutions must identify high-risk customers to avoid financial losses and maintain operational stability. The rising complexity of consumer behaviour makes traditional assessment methods less effective for modern credit environments.

Although models like logistic regression have been widely used for credit scoring, they struggle to capture the non-linear and interdependent patterns found in real-world financial data. Factors such as demographics, payment history, billing patterns, and credit utilisation interact in ways that exceed the capabilities of classical statistical approaches, often resulting in reduced accuracy and misclassification.

Machine Learning (ML) offers an effective solution by learning complex relationships within large datasets and delivering stronger predictive performance. Algorithms such as Random Forest and XGBoost have demonstrated high accuracy, scalability, and robustness in credit scoring tasks due to their ability to model intricate patterns and handle noisy, high-dimensional data.

This project uses the Taiwan Credit Card Default dataset containing 30,000 customer records with demographic, billing, and payment features. Since the dataset is highly imbalanced, SMOTE oversampling is applied to improve the representation of default cases and strengthen model performance. Based on findings from prior research, Random Forest and XGBoost are selected for evaluation using metrics such as Accuracy, Precision, Recall, F1-score, and ROC-AUC to determine the most effective model for deployment.

The goal of the system is to provide a reliable, GUI-based credit risk prediction tool that supports financial institutions in making informed decisions. By integrating machine learning into the evaluation process, the project aims to reduce default-related losses, improve risk management, and promote responsible lending in a data-driven financial ecosystem.

## 2. SUPPORTING LITERATURE

### 2.1 Literature Review

Paper 1 – Accuracy Comparison between Five Machine Learning Algorithms for Financial Risk Evaluation (Dong et al., 2024)

This study compares five classical machine learning models - k-NN, Naïve Bayes, Decision Tree, Logistic Regression, and Random Forest—for predicting loan defaults. It highlights the need for reliable models in financial risk evaluation, as minor prediction errors can lead to major losses. The authors examine the effects of preprocessing techniques such as normalization, standardization, missing value handling, and class imbalance correction with SMOTE. Experiments were conducted on a small dataset of 1,000 records and the widely used Taiwan Credit Card Default dataset of 30,000 customers.

Models were trained using an 80:20 split with grid search–based hyperparameter tuning and evaluated through Accuracy, Precision, Recall, F1-score, and ROC-AUC, supplemented by cross-validation for robustness. Results show that Random Forest consistently outperformed all other algorithms across both datasets, achieving superior accuracy and ROC-AUC and exhibiting strong resilience to imbalanced data. Logistic Regression and Decision Tree produced moderate, interpretable results, while k-NN was highly sensitive to feature scaling and Naïve Bayes struggled with correlated features. Notably, omitting SMOTE sharply reduced recall, emphasizing its importance for detecting default cases.

Contribution:

The paper reinforces the strength of ensemble methods—especially Random Forest—in financial risk assessment and shows how preprocessing choices strongly influence outcomes. Limitations include the exclusion of deep learning models and limited validation across varied financial datasets, suggesting opportunities for future studies to explore neural networks and broader financial domains.

|                      |                                                                                                                                                                                                                                                                                                                  |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TITLE                | Accuracy Comparison between Five Machine Learning Algorithms for Financial Risk Evaluation                                                                                                                                                                                                                       |
| AREA OF WORK         | Compares classical ML algorithms (k-NN, Naïve Bayes, Decision Tree, Logistic Regression, Random Forest) in financial risk evaluation focusing on loan default prediction.                                                                                                                                        |
| DATASET              | Two datasets: (1) UCI dataset with 1,000 records, (2) Taiwan Credit Card Default dataset with 30,000 records.                                                                                                                                                                                                    |
| METHODOLOGY/STRATEGY | Full ML pipeline in Scikit-learn: preprocessing (normalization, standardization, missing value imputation), class imbalance handling with SMOTE, model training using 80:20 train-test split, grid search–based hyperparameter tuning. Performance measured with Accuracy, Precision, Recall, F1-score, ROC-AUC. |
| ALGORITHM            | k-NN, Naïve Bayes, Decision Tree, Logistic Regression, Random Forest                                                                                                                                                                                                                                             |
| RESULT/PRECISION     | Random Forest outperformed all others in both Accuracy and ROC-AUC. Logistic Regression and Decision Trees performed moderately; Naïve Bayes struggled with correlated data. Without SMOTE, recall dropped significantly.                                                                                        |
| ADVANTAGES           | Demonstrates robustness of ensemble methods (Random Forest), highlights importance of preprocessing and imbalance correction for financial datasets.                                                                                                                                                             |
| FUTURE PROPOSAL      | Include deep learning methods (e.g., Neural Networks) and test across diverse datasets for generalization.                                                                                                                                                                                                       |

Table 2.1.1 Literature review of Paper 1

## Paper 2 – A Novel Framework for Enhancing Transparency in Credit Scoring: Leveraging Shapley Values for Interpretable Credit Scorecards (Hlongwane et al., 2024)

This study examines the challenge of balancing predictive accuracy and interpretability in credit scoring. Although advanced machine learning models—Random Forest, XGBoost, LightGBM, and CatBoost—outperform Logistic Regression, their “black-box” nature limits adoption in the banking sector, where regulatory standards require transparent decision-making. To address this, the authors propose a framework that combines Shapley values (SHAP) with these models to produce interpretable, scorecard-like explanations.

The framework was tested on the Taiwan Credit Card Default dataset (30,000 records) and the Home Credit dataset (356,255 records). Preprocessing involved discretization, one-hot encoding, and feature selection, followed by an 80:20 train–test split with hyperparameter tuning. SHAP values were computed for feature groups to create credit scorecards that retain the structure of traditional scoring methods while leveraging the predictive strength of advanced models.

Results show that tree-based ensembles achieve significantly higher accuracy than Logistic Regression, effectively capturing nonlinear patterns in complex financial data. With SHAP integration, these models remained interpretable, enabling clear explanations of both loan approvals and rejections, thus meeting regulatory requirements and enhancing communication with customers. This combination of accuracy and explainability makes the framework suitable for real-world credit risk assessment.

### Contribution:

The study offers a practical solution for integrating high-performing ML models into regulated financial environments by ensuring transparency through SHAP-based scorecards. While computationally demanding and dependent on well-designed features, the approach strengthens trust and compliance. Future work could explore real-time credit scoring and validate the framework across global datasets.

|                      |                                                                                                                                                                                                                                                                  |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TITLE                | A Novel Framework for Enhancing Transparency in Credit Scoring: Leveraging Shapley Values for Interpretable Credit Scorecards                                                                                                                                    |
| AREA OF WORK         | Bridges the gap between accuracy and interpretability in credit scoring by integrating SHAP with advanced ML models to create transparent scorecards.                                                                                                            |
| DATASET              | Taiwan Credit Card Default dataset (30,000 records) and Home Credit Dataset (356,255 customer records).                                                                                                                                                          |
| METHODOLOGY/STRATEGY | Preprocessing (discretization, one-hot encoding, feature selection). Models: Random Forest, XGBoost, LightGBM, CatBoost. Computed SHAP values to derive interpretable credit scorecards comparable to Logistic Regression while maintaining predictive strength. |
| ALGORITHM            | Random Forest, XGBoost, LightGBM, CatBoost with Shapley Values (SHAP)                                                                                                                                                                                            |
| RESULT/PRECISION     | Tree-based models achieved higher accuracy than Logistic Regression. With SHAP, models became interpretable and regulator-friendly, meeting transparency requirements.                                                                                           |
| ADVANTAGES           | Combines high predictive power of ensemble/boosting methods with SHAP-based interpretability, enabling regulatory compliance and better customer communication.                                                                                                  |
| FUTURE PROPOSAL      | Extend framework to real-time scoring, dynamic variable updates, and testing across diverse financial products.                                                                                                                                                  |

Table 2.1.2 Literature review of Paper 2

### Paper 3 – Interpretable Machine Learning for Imbalanced Credit Scoring Datasets (Chen et al., 2024)

This article examines how class imbalance affects not only predictive accuracy but also the stability of interpretability methods in credit scoring. Since default cases are rare in real-world datasets, models often become biased toward the majority class. While previous studies largely focus on improving prediction under imbalance, this work uniquely analyzes how imbalance influences the reliability of explanation techniques such as LIME and SHAP—an important requirement in regulated financial environments where model decisions must be transparent.

The analysis uses three datasets: UK residential mortgage data (2016–2020) from the European Datawarehouse and two open-source datasets including the Taiwan Credit Card Default dataset. Random Forest and XGBoost were selected for prediction due to their strong performance. Various imbalance levels were artificially introduced, and interpretability stability was evaluated using metrics like Sequential Rank Agreement (SRA) for feature ranking and Coefficient of Variation (CV) for feature importance values.

Results indicate that as class imbalance increases, both model accuracy and interpretability stability decline. At moderate imbalance, explanations remain usable, but at extreme imbalance (default rates below 5%), LIME and SHAP produce inconsistent feature rankings, posing risks for regulatory compliance and customer communication. Resampling techniques such as SMOTE and hybrid sampling improved both predictive performance and explanation consistency, demonstrating that preprocessing strongly influences interpretability.

#### Contribution:

This study highlights that class imbalance weakens not only classification performance but also the reliability of model explanations. It introduces an experimental framework for evaluating interpretability stability under varying imbalance levels—valuable for high-stakes credit risk applications. Limitations include the reliance on post-hoc interpretability methods; future work could integrate imbalance handling directly into inherently interpretable models or test across a broader range of financial products.

|                      |                                                                                                                                                                                                                                                                         |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TITLE                | Interpretable Machine Learning for Imbalanced Credit Scoring Datasets                                                                                                                                                                                                   |
| AREA OF WORK         | Explores how class imbalance affects not only predictive accuracy but also interpretability stability of credit scoring models using LIME and SHAP.                                                                                                                     |
| DATASET              | UK Residential Mortgage dataset (2016–2020, European Datawarehouse), Taiwan Credit Card Default dataset, South German Credit dataset.                                                                                                                                   |
| METHODOLOGY/STRATEGY | Applied Random Forest and XGBoost as predictive models. Assessed interpretability stability under varying imbalance levels using Sequential Rank Agreement (SRA) and Coefficient of Variation (CV). Resampling methods (SMOTE, hybrid sampling) tested for improvement. |
| ALGORITHM            | Random Forest, XGBoost with interpretability tools: LIME, SHAP                                                                                                                                                                                                          |
| RESULT/PRECISION     | As imbalance increased, predictive performance and interpretability stability declined. At extreme imbalance (<5% defaults), explanations became inconsistent. Resampling improved both accuracy and explanation reliability.                                           |
| ADVANTAGES           | Provides first systematic study linking class imbalance to interpretability stability. Offers an evaluation framework for LIME/SHAP under imbalance.                                                                                                                    |
| FUTURE PROPOSAL      | Develop interpretability methods inherently robust to imbalance and integrate balancing techniques directly into interpretable models.                                                                                                                                  |

Table 2.1.3 Literature review of Paper 3

## 2.2 Literature Review Summary

Credit risk prediction has evolved from manual assessment and traditional statistical models toward advanced machine learning techniques. Logistic regression and other simple models struggle to capture the complexity of financial risk, increasing the need for automated systems that are both accurate and interpretable. Recent research highlights three major directions: benchmarking classical ML models, enhancing ensemble models with interpretability tools, and addressing class imbalance challenges.

Dong et al. (2024) compared five classical algorithms—k-NN, Naïve Bayes, Decision Tree, Logistic Regression, and Random Forest—using the Taiwan Credit Card Default dataset and a small UCI dataset. Random Forest consistently achieved the highest accuracy and ROC-AUC, with SMOTE significantly improving recall for minority (defaulter) cases. Their findings reinforce the superiority of ensemble methods for financial risk evaluation.

Hlongwane et al. (2024) tackled the tension between predictive performance and interpretability. While models like XGBoost, Random Forest, and LightGBM deliver strong performance, their black-box nature limits adoption in regulated industries. The authors introduced a SHAP-based credit scorecard framework that preserves accuracy while providing regulator-friendly explanations. Tests on the Taiwan Credit and Home Credit datasets showed that SHAP-enhanced ensembles can maintain transparency without compromising performance.

Chen et al. (2024) investigated how class imbalance affects both prediction and interpretability. Using datasets such as UK mortgage data and the Taiwan Credit dataset, they found that higher imbalance leads to unstable explanations from LIME and SHAP, especially when default rates drop below 5%. Applying resampling methods like SMOTE improved both classification accuracy and interpretability stability, demonstrating the importance of imbalance handling for trustworthy model explanations.

Collectively, these studies show that modern credit risk prediction benefits from ensemble methods like Random Forest and XGBoost, supported by SMOTE for imbalance correction and SHAP for interpretability. This combination enables the development of accurate, transparent, and regulator-compliant credit scoring systems.

| TITLE                                                                                                                                                                           | SUMMARY                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dong H., Liu R., Tham A.W. (2024) <i>Accuracy Comparison between Five Machine Learning Algorithms for Financial Risk Evaluation</i>                                             | Benchmarks k-NN, Naïve Bayes, Decision Tree, Logistic Regression, and Random Forest on a UCI dataset (1,000 records) and the Taiwan Credit dataset (30,000 records). Preprocessing included normalization, standardization, and SMOTE. Random Forest achieved the highest accuracy and ROC-AUC; k-NN was sensitive to scaling, and Naïve Bayes struggled with correlated features. Highlights ensemble models as more robust for financial risk prediction. |
| Hlongwane R., Ramabao K., Mongwe W. (2024) <i>A Novel Framework for Enhancing Transparency in Credit Scoring: Leveraging Shapley Values for Interpretable Credit Scorecards</i> | Addresses the trade-off between accuracy and interpretability. Using the Taiwan Credit and Home Credit datasets, the authors applied RF, XGBoost, LightGBM, and CatBoost with SHAP to create interpretable scorecards. Achieved high accuracy with regulator-friendly explanations, showing ensembles can be both powerful and transparent, though computationally demanding.                                                                               |
| Chen Y., Calabrese R., Martin-Barragan B. (2024) <i>Interpretable Machine Learning for Imbalanced Credit Scoring Datasets</i>                                                   | Examines how class imbalance affects prediction and interpretability using UK mortgage data and the Taiwan Credit dataset. RF and XGBoost with LIME and SHAP showed declining accuracy and explanation stability as imbalance increased. SMOTE improved both performance and interpretability consistency, emphasizing the need for imbalance handling in reliable credit scoring.                                                                          |

Table 2.2.1 Summary of Literatures

## 2.3 Findings and Proposals

The Taiwan Credit Card Default dataset initially contained 30,000 records with 24 features. After correcting invalid categorical values and removing duplicates, 29,965 valid records remained. The target variable distribution—78% non-defaults and 22% defaults—confirmed a realistic class imbalance. Because this imbalance reflects real-world credit behavior, no resampling techniques (e.g., SMOTE) were applied; instead, models were trained directly on the natural distribution.

Preprocessing showed no missing values, although inconsistencies in features such as education and marital status were corrected. Financial attributes such as bill amounts and payment amounts contained outliers, which were retained but scaled using RobustScaler to prevent distortion during training. Correlation analysis revealed redundancy among several bill and payment variables, while Random Forest-based feature importance identified PAY\_0, LIMIT\_BAL, AGE, and multiple billing/payment features as strong predictors.

Given these insights and findings from prior literature, tree-based ensemble models such as Random Forest and XGBoost were selected for predictive modeling due to their robustness in financial classification tasks. Evaluation focuses on precision, recall, F1-score, and ROC-AUC, as these metrics better account for the minority default class compared to accuracy alone. To support transparency, SHAP explanations will be integrated to provide clear reasoning behind each prediction, addressing interpretability requirements for financial decision-making. The final phase involves deploying the best-performing model in a GUI-based application to enable real-time default prediction for analysts and lenders.

Future work may incorporate additional boosting models such as LightGBM and CatBoost, explore advanced feature selection strategies, and further enhance interpretability frameworks. Overall, the approach aims to deliver a reliable, interpretable, and practically deployable credit risk prediction system.

### 3. SYSTEM ANALYSIS

#### 3.1 Analysis of the dataset

##### 3.1.1 About the dataset

The Taiwan Credit Card Default dataset, obtained from the UCI Machine Learning Repository, is a widely used benchmark dataset for credit risk prediction and loan default analysis. It contains information on credit card clients in Taiwan, focusing on demographic details, credit limits, repayment behaviour, and default status.

Dataset Link: <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>

Size: The dataset consists of 30,000 client records with 24 features and one target variable indicating whether the client defaulted on the next month's payment (1 = default, 0 = non-default). After preprocessing and cleaning (removal of duplicates and correction of invalid categorical codes), the working dataset includes 29,965 valid records.

Data Structure: The dataset contains structured variables covering demographic attributes (e.g., age, sex, education, marriage), financial attributes (e.g., credit limit, bill amounts, payment amounts), and repayment history (e.g., PAY\_0 to PAY\_6). The target variable represents default status. Outliers present in financial attributes were scaled using RobustScaler to preserve the real distribution while minimizing skewness.

Applications: The dataset is primarily used for building credit scoring models to classify customers as high or low risk of default. Its rich combination of demographic, financial, and behavioral attributes makes it suitable for developing predictive models using machine learning. Additionally, it serves as a benchmark for evaluating imbalance-handling strategies, feature selection techniques, and explainable AI frameworks such as SHAP and LIME in the context of financial decision-making.

|    | B         | C      | D         | E        | F   | G     | H     | I     | J     | K     | L     | M     | N         | O         | P         | Q         | R         | S         | T        | U        | V        | W        | X        | Y        | Z       | AA                 |
|----|-----------|--------|-----------|----------|-----|-------|-------|-------|-------|-------|-------|-------|-----------|-----------|-----------|-----------|-----------|-----------|----------|----------|----------|----------|----------|----------|---------|--------------------|
| 2  | LIMIT_BAL | SEX    | EDUCATION | MARRIAGE | AGE | PAY_0 | PAY_1 | PAY_2 | PAY_3 | PAY_4 | PAY_5 | PAY_6 | BILL_AMT1 | BILL_AMT2 | BILL_AMT3 | BILL_AMT4 | BILL_AMT5 | BILL_AMT6 | PAY_AMT1 | PAY_AMT2 | PAY_AMT3 | PAY_AMT4 | PAY_AMT5 | PAY_AMT6 | default | payment next month |
| 3  | 1         | 20000  | 2         | 2        | 1   | 24    | 2     | 2     | -1    | -1    | -2    | -2    | 3913      | 3102      | 689       | 0         | 0         | 0         | 689      | 0        | 0        | 0        | 0        | 0        | 1       |                    |
| 4  | 2         | 120000 | 2         | 2        | 2   | 26    | -1    | 2     | 0     | 0     | 0     | 0     | 2         | 2682      | 1725      | 2682      | 3272      | 3455      | 3261     | 0        | 1000     | 1000     | 1000     | 0        | 2000    | 1                  |
| 5  | 3         | 90000  | 2         | 2        | 2   | 34    | 0     | 0     | 0     | 0     | 0     | 0     | 0         | 29239     | 14027     | 13559     | 14331     | 14948     | 15549    | 1518     | 1500     | 1000     | 1000     | 1000     | 5000    | 0                  |
| 6  | 4         | 50000  | 2         | 2        | 1   | 37    | 0     | 0     | 0     | 0     | 0     | 0     | 0         | 46990     | 48233     | 48291     | 28314     | 28959     | 29547    | 2000     | 2019     | 1200     | 1100     | 1069     | 1000    | 0                  |
| 7  | 5         | 50000  | 1         | 2        | 1   | 57    | -1    | 0     | -1    | 0     | 0     | 0     | 0         | 8617      | 5670      | 35835     | 20940     | 19146     | 19131    | 2000     | 36681    | 10000    | 9000     | 689      | 679     | 0                  |
| 8  | 6         | 50000  | 1         | 1        | 2   | 37    | 0     | 0     | 0     | 0     | 0     | 0     | 0         | 64400     | 57069     | 57608     | 19394     | 19619     | 20024    | 2500     | 1815     | 657      | 1000     | 1000     | 800     | 0                  |
| 9  | 7         | 500000 | 1         | 1        | 2   | 29    | 0     | 0     | 0     | 0     | 0     | 0     | 0         | 367965    | 412023    | 445007    | 542653    | 483003    | 473944   | 55000    | 40000    | 38000    | 20239    | 13750    | 13770   | 0                  |
| 10 | 8         | 100000 | 2         | 2        | 2   | 23    | 0     | -1    | -1    | 0     | 0     | -1    | 11876     | 380       | 601       | 221       | -159      | 567       | 380      | 601      | 0        | 581      | 1687     | 1542     | 0       | 0                  |
| 11 | 9         | 140000 | 2         | 3        | 1   | 28    | 0     | 0     | 2     | 0     | 0     | 0     | 11285     | 14096     | 12108     | 12211     | 11793     | 3719      | 3329     | 0        | 432      | 1000     | 1000     | 1000     | 0       | 0                  |
| 12 | 10        | 20000  | 1         | 3        | 2   | 35    | -2    | -2    | -2    | -2    | -1    | -1    | 0         | 0         | 0         | 0         | 13007     | 13912     | 0        | 0        | 0        | 13007    | 1122     | 0        | 0       | 0                  |
| 13 | 11        | 200000 | 2         | 3        | 2   | 34    | 0     | 0     | 2     | 0     | 0     | -1    | 11073     | 9787      | 5535      | 2513      | 1828      | 3731      | 2306     | 12       | 50       | 300      | 3738     | 66       | 0       | 0                  |
| 14 | 12        | 260000 | 2         | 1        | 2   | 51    | -1    | -1    | -1    | -1    | -1    | 2     | 12261     | 21670     | 9966      | 8517      | 22287     | 13668     | 21818    | 9966     | 8583     | 22301    | 0        | 3640     | 0       |                    |
| 15 | 13        | 630000 | 2         | 2        | 2   | 41    | -1    | 0     | -1    | -1    | -1    | -1    | 1         | 12137     | 6500      | 6500      | 6500      | 6500      | 2870     | 1000     | 6500     | 6500     | 6500     | 2870     | 0       | 0                  |
| 16 | 14        | 70000  | 1         | 2        | 2   | 30    | 1     | 2     | 2     | 0     | 0     | 0     | 2         | 65802     | 67369     | 65701     | 66782     | 36137     | 36894    | 3200     | 0        | 3000     | 3000     | 1500     | 0       | 1                  |
| 17 | 15        | 250000 | 1         | 1        | 2   | 29    | 0     | 0     | 0     | 0     | 0     | 0     | 0         | 70887     | 67060     | 63561     | 59696     | 56875     | 55512    | 3000     | 3000     | 3000     | 3000     | 3000     | 3000    | 0                  |
| 18 | 16        | 50000  | 2         | 3        | 3   | 23    | 1     | 2     | 0     | 0     | 0     | 0     | 0         | 50614     | 29173     | 28116     | 28771     | 29531     | 30211    | 0        | 1500     | 1100     | 1200     | 1300     | 1100    | 0                  |
| 19 | 17        | 20000  | 1         | 1        | 2   | 24    | 0     | 0     | 2     | 2     | 2     | 2     | 2         | 15376     | 18010     | 17428     | 18338     | 17905     | 19104    | 3200     | 0        | 1500     | 0        | 1650     | 0       | 1                  |
| 20 | 18        | 320000 | 1         | 1        | 1   | 49    | 0     | 0     | 0     | -1    | -1    | -1    | 1         | 253286    | 246536    | 194663    | 70074     | 5856      | 195599   | 10358    | 10000    | 75940    | 20000    | 195599   | 50000   | 0                  |
| 21 | 19        | 360000 | 2         | 1        | 1   | 49    | 1     | -2    | -2    | -2    | -2    | -2    | 0         | 0         | 0         | 0         | 0         | 0         | 0        | 0        | 0        | 0        | 0        | 0        | 0       | 0                  |
| 22 | 20        | 180000 | 2         | 1        | 2   | 29    | 1     | -2    | -2    | -2    | -2    | -2    | 0         | 0         | 0         | 0         | 0         | 0         | 0        | 0        | 0        | 0        | 0        | 0        | 0       | 0                  |
| 23 | 21        | 130000 | 2         | 3        | 2   | 39    | 0     | 0     | 0     | 0     | 0     | -1    | 38358     | 27688     | 24489     | 20616     | 11802     | 930       | 3000     | 1537     | 1000     | 2000     | 930      | 33764    | 0       | 0                  |
| 24 | 22        | 120000 | 2         | 2        | 1   | 39    | -1    | -1    | -1    | -1    | -1    | -1    | -1        | 316       | 316       | 316       | 0         | 632       | 316      | 316      | 316      | 0        | 632      | 316      | 0       | 1                  |
| 25 | 23        | 70000  | 2         | 2        | 2   | 26    | 2     | 0     | 0     | 0     | 2     | 2     | 2         | 41087     | 42445     | 45020     | 44006     | 46905     | 46012    | 2007     | 3582     | 0        | 3601     | 0        | 1820    | 1                  |
| 26 | 24        | 450000 | 2         | 1        | 1   | 40    | -2    | -2    | -2    | -2    | -2    | -2    | -2        | 5512      | 19420     | 1473      | 560       | 0         | 0        | 19428    | 1473     | 560      | 0        | 0        | 1128    | 1                  |
| 27 | 25        | 90000  | 1         | 1        | 2   | 23    | 0     | 0     | 0     | 0     | -1    | 0     | 0         | 4744      | 7070      | 0         | 5398      | 6360      | 8292     | 5757     | 0        | 5398     | 1200     | 2045     | 2000    | 0                  |
| 28 | 26        | 50000  | 1         | 3        | 2   | 23    | 0     | 0     | 0     | 0     | 0     | 0     | 0         | 47620     | 41810     | 36023     | 28967     | 29829     | 30046    | 1973     | 1426     | 1001     | 1432     | 1062     | 997     | 0                  |
| 29 | 27        | 60000  | 1         | 1        | 2   | 27    | 1     | -2    | -1    | -1    | -1    | -1    | -1        | -109      | -425      | 259       | -57       | 127       | -189     | 0        | 1000     | 0        | 500      | 0        | 1000    | 1                  |
| 30 | 28        | 50000  | 2         | 3        | 2   | 30    | 0     | 0     | 0     | 0     | 0     | 0     | 0         | 22541     | 16138     | 17163     | 17878     | 18931     | 19617    | 1300     | 1300     | 1000     | 1500     | 1000     | 1012    | 0                  |
| 31 | 29        | 50000  | 2         | 3        | 1   | 47    | -1    | -1    | -1    | -1    | -1    | -1    | -1        | 650       | 3415      | 3416      | 2040      | 30430     | 257      | 3415     | 3421     | 2044     | 30430    | 257      | 0       | 0                  |
| 32 | 30        | 50000  | 1         | 1        | 2   | 26    | 0     | 0     | 0     | 0     | 0     | 0     | 0         | 15329     | 16575     | 17496     | 17907     | 18375     | 11400    | 1500     | 1500     | 1000     | 1000     | 1600     | 0       | 0                  |

Fig 3.1.1 Sample Dataset

### 3.1.2 Explore the dataset

#### a) Loading the Dataset

The dataset was loaded into the Python environment using Pandas for further analysis. After successful import, it was confirmed that the dataset contained 30,000 rows and 25 columns, with 24 independent features describing demographic, credit, and repayment information, along with one dependent variable representing the default status. This ensured that the dataset was correctly read and ready for exploration.

```
# Load dataset (already cleaned header)
df = pd.read_csv("/content/default of credit card clients.csv", header=1)
df = df.rename(columns={"default payment next month": "DEFAULT"}).drop(columns=["ID"], errors="ignore")
```

Fig 3.1.2 Loading dataset and renaming target variable

#### b) Dataset Information (info())

The info() function was applied to examine the structure of the dataset. It verified that all 25 columns were of numerical types (integer or float), which makes them directly compatible with machine learning algorithms. The output also confirmed that there were no null values across any of the features, indicating strong data completeness.

```
[ ] # info
print("\nInfo:\n", df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 24 columns):
#   Column      Non-Null Count  Dtype
---  -
0   LIMIT_BAL   30000 non-null  int64
1   SEX         30000 non-null  int64
2   EDUCATION   30000 non-null  int64
3   MARRIAGE    30000 non-null  int64
4   AGE         30000 non-null  int64
5   PAY_0       30000 non-null  int64
6   PAY_2       30000 non-null  int64
7   PAY_3       30000 non-null  int64
8   PAY_4       30000 non-null  int64
9   PAY_5       30000 non-null  int64
10  PAY_6       30000 non-null  int64
11  BILL_AMT1   30000 non-null  int64
12  BILL_AMT2   30000 non-null  int64
13  BILL_AMT3   30000 non-null  int64
14  BILL_AMT4   30000 non-null  int64
15  BILL_AMT5   30000 non-null  int64
16  BILL_AMT6   30000 non-null  int64
17  PAY_AMT1    30000 non-null  int64
18  PAY_AMT2    30000 non-null  int64
19  PAY_AMT3    30000 non-null  int64
20  PAY_AMT4    30000 non-null  int64
21  PAY_AMT5    30000 non-null  int64
22  PAY_AMT6    30000 non-null  int64
23  DEFAULT     30000 non-null  int64
dtypes: int64(24)
memory usage: 5.5 MB
```

Fig 3.1.3 Information about the dataset

### c) Dataset Dimensions (shape)

The shape function was used to check the size of the dataset, which returned (30000, 25). This validated that the dataset has a sufficiently large number of samples and features to support robust statistical analysis and machine learning modeling.

```
# Shape
print("Shape:", df.shape)
```

```
Shape: (30000, 24)
```

Fig 3.1.4 Shape of the dataset

### d) Data Types (dtypes)

The dtypes attribute was examined to identify the type of data stored in each column. All variables were confirmed as either integers or floats, covering both categorical codes (e.g., SEX, EDUCATION, MARRIAGE) and continuous variables (e.g., LIMIT\_BAL, BILL\_AMT,

PAY\_AMT, AGE). This step confirmed that the dataset is structurally consistent and ready for preprocessing.

```
# dtypes
print("\nDtypes:\n", df.dtypes)
```

Dtypes:

|           |       |
|-----------|-------|
| LIMIT_BAL | int64 |
| SEX       | int64 |
| EDUCATION | int64 |
| MARRIAGE  | int64 |
| AGE       | int64 |
| PAY_0     | int64 |
| PAY_2     | int64 |
| PAY_3     | int64 |
| PAY_4     | int64 |
| PAY_5     | int64 |
| PAY_6     | int64 |
| BILL_AMT1 | int64 |
| BILL_AMT2 | int64 |
| BILL_AMT3 | int64 |
| BILL_AMT4 | int64 |
| BILL_AMT5 | int64 |
| BILL_AMT6 | int64 |
| PAY_AMT1  | int64 |
| PAY_AMT2  | int64 |
| PAY_AMT3  | int64 |
| PAY_AMT4  | int64 |
| PAY_AMT5  | int64 |
| PAY_AMT6  | int64 |
| DEFAULT   | int64 |

dtype: object

Fig 3.1.5 Data type of the dataset

#### e) Null Value Check

A null-value inspection was carried out using `isnull().sum()`, and the results confirmed that there were no missing values in any of the features. This means that the dataset does not require imputation for handling incomplete records, simplifying the preprocessing stage.

```
# Check nulls
print("\nMissing values:\n", df.isna().sum())
print("\nTotal nulls:", df.isnull().sum().sum())
```

Missing values:

|           |   |
|-----------|---|
| LIMIT_BAL | 0 |
| SEX       | 0 |
| EDUCATION | 0 |
| MARRIAGE  | 0 |
| AGE       | 0 |
| PAY_0     | 0 |
| PAY_2     | 0 |
| PAY_3     | 0 |
| PAY_4     | 0 |
| PAY_5     | 0 |
| PAY_6     | 0 |
| BILL_AMT1 | 0 |
| BILL_AMT2 | 0 |
| BILL_AMT3 | 0 |
| BILL_AMT4 | 0 |
| BILL_AMT5 | 0 |
| BILL_AMT6 | 0 |
| PAY_AMT1  | 0 |
| PAY_AMT2  | 0 |
| PAY_AMT3  | 0 |
| PAY_AMT4  | 0 |
| PAY_AMT5  | 0 |
| PAY_AMT6  | 0 |
| DEFAULT   | 0 |

dtype: int64

Total nulls: 0

Fig 3.1.6 Checking Null Values of the dataset

## f) Duplicate Check

The dataset was checked for duplicate records. A small number of duplicates were detected, which can introduce redundancy and bias in modeling. These duplicates will be removed during preprocessing to ensure that the dataset represents unique client records.

```
# Check duplicates
print("\nDuplicate rows:", df.duplicated().sum())
```

Duplicate rows: 35

Fig 3.1.7 Checking Duplicate Values of the dataset

## g) Summary Statistics (describe())

The describe() function provided descriptive statistics for the numerical variables, including mean, median, standard deviation, minimum, maximum, and quartiles. Financial attributes such as LIMIT\_BAL, BILL\_AMT, and PAY\_AMT displayed high variance and right-skewed distributions, reflecting real-world credit and repayment behavior with the presence of extreme values. Demographic features such as AGE showed a reasonable spread within expected ranges, while categorical variables coded numerically (e.g., EDUCATION, MARRIAGE) indicated some inconsistencies that will be corrected during data cleaning. These insights confirmed that while the dataset is generally clean, careful handling of outliers and categorical inconsistencies will be necessary.

```
# Summary statistics
print("\nSummary statistics:\n", df.describe().T)
```

|           | count   | mean          | std           | min       | 25%      | 50%      | 75%       | max       |
|-----------|---------|---------------|---------------|-----------|----------|----------|-----------|-----------|
| LIMIT_BAL | 30000.0 | 167484.322667 | 129747.661567 | 10000.0   | 50000.00 |          |           |           |
| SEX       | 30000.0 | 1.603733      | 0.489129      | 1.0       | 1.00     | 140000.0 | 240000.00 | 1000000.0 |
| EDUCATION | 30000.0 | 1.853133      | 0.790349      | 0.0       | 1.00     | 2.0      | 2.00      | 2.0       |
| MARRIAGE  | 30000.0 | 1.551867      | 0.521970      | 0.0       | 1.00     | 2.0      | 2.00      | 6.0       |
| AGE       | 30000.0 | 35.485500     | 9.217904      | 21.0      | 28.00    | 34.0     | 41.00     | 79.0      |
| PAY_0     | 30000.0 | -0.016700     | 1.123802      | -2.0      | -1.00    | 0.0      | 0.00      | 8.0       |
| PAY_2     | 30000.0 | -0.133767     | 1.197186      | -2.0      | -1.00    | 0.0      | 0.00      | 8.0       |
| PAY_3     | 30000.0 | -0.166200     | 1.196868      | -2.0      | -1.00    | 0.0      | 0.00      | 8.0       |
| PAY_4     | 30000.0 | -0.220667     | 1.169139      | -2.0      | -1.00    | 0.0      | 0.00      | 8.0       |
| PAY_5     | 30000.0 | -0.266200     | 1.133187      | -2.0      | -1.00    | 0.0      | 0.00      | 8.0       |
| PAY_6     | 30000.0 | -0.291100     | 1.149988      | -2.0      | -1.00    | 0.0      | 0.00      | 8.0       |
| BILL_AMT1 | 30000.0 | 51223.330900  | 73635.860576  | -165580.0 | 3558.75  | 22381.5  | 67091.00  | 964511.0  |
| BILL_AMT2 | 30000.0 | 49179.075167  | 71173.768783  | -69777.0  | 2984.75  | 21200.0  | 64006.25  | 983931.0  |
| BILL_AMT3 | 30000.0 | 47013.154800  | 69349.387427  | -157264.0 | 2666.25  | 20088.5  | 60164.75  | 1664089.0 |
| BILL_AMT4 | 30000.0 | 43262.948967  | 64332.856134  | -170000.0 | 2326.75  | 19052.0  | 54506.00  | 891586.0  |
| BILL_AMT5 | 30000.0 | 40311.400967  | 60797.155770  | -81334.0  | 1763.00  | 18104.5  | 50190.50  | 927171.0  |
| BILL_AMT6 | 30000.0 | 38871.760400  | 59554.107537  | -339603.0 | 1256.00  | 17071.0  | 49198.25  | 961664.0  |
| PAY_AMT1  | 30000.0 | 5663.580500   | 16563.280354  | 0.0       | 1000.00  | 2100.0   | 5006.00   | 873552.0  |
| PAY_AMT2  | 30000.0 | 5921.163500   | 23040.870402  | 0.0       | 833.00   | 2009.0   | 5000.00   | 1684259.0 |
| PAY_AMT3  | 30000.0 | 5225.681500   | 17606.961470  | 0.0       | 390.00   | 1800.0   | 4505.00   | 896040.0  |
| PAY_AMT4  | 30000.0 | 4826.076867   | 15666.159744  | 0.0       | 296.00   | 1500.0   | 4013.25   | 621000.0  |
| PAY_AMT5  | 30000.0 | 4799.387633   | 15278.305679  | 0.0       | 252.50   | 1500.0   | 4031.50   | 426529.0  |
| PAY_AMT6  | 30000.0 | 5215.502567   | 17777.465775  | 0.0       | 117.75   | 1500.0   | 4000.00   | 528666.0  |
| DEFAULT   | 30000.0 | 0.221200      | 0.415062      | 0.0       | 0.00     | 0.0      | 0.00      | 1.0       |

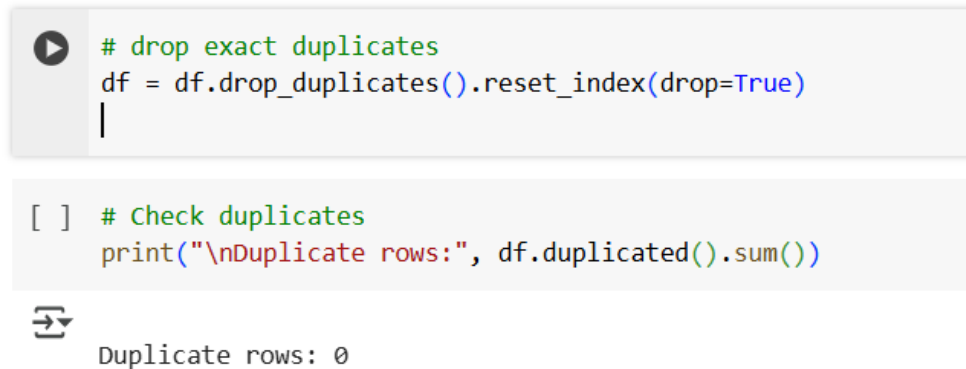
Fig 3.1.8 Checking Summary Statistics of the dataset

## 3.2 Data Preprocessing

### 3.2.1 Data Cleaning

#### a) Drop Exact Duplicates

When checking for duplicates, a small number of rows were found to be identical. These records were dropped using `drop_duplicates()` and the dataset index was reset. This ensured that each client is represented uniquely, preventing bias from repeated entries during model training.



```
# drop exact duplicates
df = df.drop_duplicates().reset_index(drop=True)

[ ] # Check duplicates
print("\nDuplicate rows:", df.duplicated().sum())
```

Duplicate rows: 0

Fig 3.2.1 Drop Exact Duplicates From the dataset

#### b) Outlier Detection

Outlier detection using the Interquartile Range (IQR) method across financial attributes (LIMIT\_BAL, BILL\_AMT, PAY\_AMT) revealed a large number of extreme values. For example, credit limit and payment amounts showed unusually high values compared to the median. Since these outliers represent realistic variations in client financial behavior, they were retained in the dataset. To minimize their impact on model sensitivity, `RobustScaler` was applied to rescale financial variables, which reduces the effect of extreme values without removing them.

```
# outlier detection (1.5*IQR) - quick counts per numeric column
num_cols = ['LIMIT_BAL', 'AGE'] + [f'BILL_AMT{i}' for i in range(1,7)] + [f'PAY_AMT{i}' for i in range(1,7)]
for c in num_cols:
    s = df[c].dropna()
    q1, q3 = s.quantile(0.25), s.quantile(0.75)
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    print(c, "outliers:", ((s < low) | (s > high)).sum())
```

```
LIMIT_BAL outliers: 167
AGE outliers: 272
BILL_AMT1 outliers: 2386
BILL_AMT2 outliers: 2390
BILL_AMT3 outliers: 2469
BILL_AMT4 outliers: 2616
BILL_AMT5 outliers: 2724
BILL_AMT6 outliers: 2688
PAY_AMT1 outliers: 2742
PAY_AMT2 outliers: 2723
PAY_AMT3 outliers: 2596
PAY_AMT4 outliers: 2993
PAY_AMT5 outliers: 2944
PAY_AMT6 outliers: 2960
```

Fig 3.2.2 Outlier Detection of the dataset

### c) Creation of Aggregate Payment Behavior Feature

A new feature `Payment_Value` was generated by summing all monthly repayment status variables:

```
df_fr['Payment_Value'] = df_fr['PAY_SEPT'] + df_fr['PAY_AUG'] +
df_fr['PAY_JUL'] + df_fr['PAY_JUN'] + df_fr['PAY_MAY'] + df_fr['PAY_APR']
```

```
display(df_fr[['Payment_Value']].head())
```

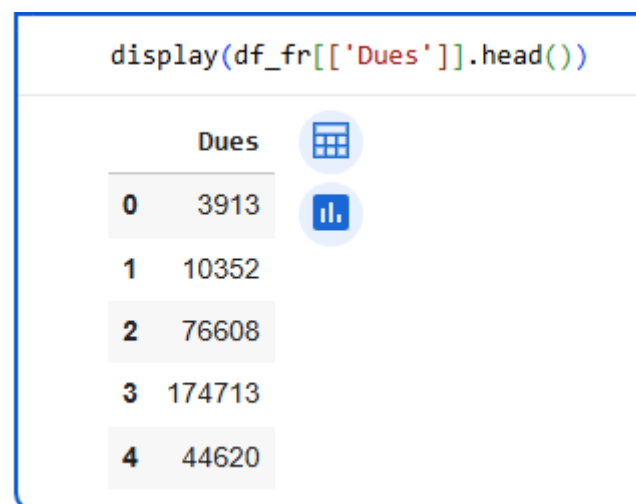
|   | Payment_Value |
|---|---------------|
| 0 | -2            |
| 1 | 3             |
| 2 | 0             |
| 3 | 0             |
| 4 | -2            |

Fig 3.2.3 A new feature `Payment_Value`

## d) Derivation of Total Outstanding Dues

Another engineered feature Dues was created to measure the difference between total bill amount and total payment amount across multiple months:

```
df_fr['Dues'] =
(df_fr['BILL_AMT_APR'] + df_fr['BILL_AMT_MAY'] + df_fr['BILL_AMT_JUN'] +
df_fr['BILL_AMT_JUL'] + df_fr['BILL_AMT_AUG'] + df_fr['BILL_AMT_SEPT']) -
(df_fr['PAY_AMT_APR'] + df_fr['PAY_AMT_MAY'] + df_fr['PAY_AMT_JUN'] +
df_fr['PAY_AMT_JUL'] + df_fr['PAY_AMT_AUG'] + df_fr['PAY_AMT_SEPT'])
```



The screenshot shows a Jupyter Notebook cell with the following code and output:

```
display(df_fr[['Dues']].head())
```

|   | Dues   |
|---|--------|
| 0 | 3913   |
| 1 | 10352  |
| 2 | 76608  |
| 3 | 174713 |
| 4 | 44620  |

Fig 3.2.4 A new feature Dues

## e) Encoding Categorical Variables

To make categorical features interpretable by ML algorithms:

- One-Hot Encoding was applied to:
  - EDUCATION
  - MARRIAGE
  - Payment status features (PAY\_SEPT ... PAY\_APR)

This created separate binary columns representing each category, preserving full information.

- Label Encoding was used for:
  - SEX (FEMALE → 0, MALE → 1)

This combination of encoding techniques ensures categorical variables are properly represented for model training.

```

▶ encoded_column_prefixes = [
    'EDUCATION_', 'MARRIAGE_',
    'PAY_SEPT_', 'PAY_AUG_', 'PAY_JUL_', 'PAY_JUN_', 'PAY_MAY_', 'PAY_APR_'
]

encoded_features_df = df_fr.filter(regex='|'.join(encoded_column_prefixes))

display(encoded_features_df.head())

```

|   | EDUCATION_1 | EDUCATION_2 | EDUCATION_3 | EDUCATION_4 | MARRIAGE_1 | MARRIAGE_2 | MARRIAGE_3 | PAY_SEPT_-1 | PAY_SEPT_0 | PAY_SEPT_1 | ... |
|---|-------------|-------------|-------------|-------------|------------|------------|------------|-------------|------------|------------|-----|
| 0 | False       | True        | False       | False       | True       | False      | False      | False       | False      | False      | ... |
| 1 | False       | True        | False       | False       | False      | True       | False      | True        | False      | False      | ... |
| 2 | False       | True        | False       | False       | False      | True       | False      | False       | True       | False      | ... |
| 3 | False       | True        | False       | False       | True       | False      | False      | False       | True       | False      | ... |
| 4 | False       | True        | False       | False       | True       | False      | False      | True        | False      | False      | ... |

5 rows x 67 columns

Fig 3.2.5 Encoding Categorical Variables

#### f) Removal of Non-Informative Features

The ID column was removed since it contains no predictive value and could introduce bias or noise into the model.

```

# Drop the original ID column
df_fr.drop('ID', axis=1, inplace=True)

```

Fig 3.2.6 Removal of Non-Informative Features

#### g) Scaling with RobustScaler

Scaling was performed on all numeric financial variables (credit limit, bill amounts, payment amounts, age). The RobustScaler transformed these features by reducing the influence of outliers and bringing all values to a more uniform scale. This is especially important for distance-based and gradient-based algorithms, which can otherwise be dominated by variables with large ranges.

```
[ ] # scale using RobustScaler (no deletions)
    scaler = RobustScaler()
    df[num_cols] = scaler.fit_transform(df[num_cols])
```

|   | LIMIT_BAL | SEX  | AGE       | BILL_AMT_SEPT | BILL_AMT_AUG | BILL_AMT_JUL | BILL_AMT_JUN | BILL_AMT_MAY | BILL_AMT_APR | PAY_AMT_SEPT | ... |
|---|-----------|------|-----------|---------------|--------------|--------------|--------------|--------------|--------------|--------------|-----|
| 0 | -0.588235 | 0.0  | -0.769231 | -0.287879     | -0.300937    | -0.343868    | -0.379736    | -0.385923    | -0.374609    | -0.523629    | ... |
| 1 | 0.000000  | 0.0  | -0.615385 | -0.308077     | -0.324390    | -0.308397    | -0.314872    | -0.313346    | -0.305492    | -0.523629    | ... |
| 2 | -0.176471 | 0.0  | 0.000000  | 0.127659      | -0.114863    | -0.114813    | -0.095640    | -0.071916    | -0.045050    | -0.126195    | ... |
| 3 | -0.411765 | 0.0  | 0.230769  | 0.418910      | 0.467731     | 0.521132     | 0.181557     | 0.222408     | 0.251636     | 0.000000     | ... |
| 4 | -0.411765 | -1.0 | 1.769231  | -0.210698     | -0.257199    | 0.281648     | 0.035376     | 0.016270     | 0.030870     | 0.000000     | ... |

5 rows x 82 columns

Fig 3.2.7 Scaling with RobustScaler the dataset

#### h) Class Imbalance

The original dataset contained a significant class imbalance between defaulters and non-defaulters. Since most customers did not default on payments, the minority class (defaulters) had far fewer samples. Training a model on such imbalanced data can lead to biased predictions, where the model learns to favor the majority class and fails to correctly identify defaulters.

To address this, Synthetic Minority Over-sampling Technique (SMOTE) was applied.

SMOTE (Synthetic Minority Oversampling Technique)

- SMOTE generates synthetic samples for the minority class instead of simply duplicating existing ones.
- New samples are created by interpolating between minority class neighbors.
- This expands the dataset in a realistic way without causing overfitting.

By balancing the dataset through SMOTE, the learning process becomes more sensitive to patterns associated with default behavior, ensuring that the model does not simply predict the majority class. This leads to improved recall and F1-score for the defaulter category, which is crucial in credit risk prediction where missing a potential defaulter can result in significant financial loss. Furthermore, using SMOTE helps the model generalize better across unseen data, providing a more reliable and fair assessment of credit risk.

```
# Apply SMOTE for balancing
smote = SMOTE(random_state=42) # Added random_state for reproducibility
x_smote, y_smote = smote.fit_resample(df.iloc[:, :-1], df['IsDefaulter'])

# Create balanced DataFrame
columns = list(df.columns)
columns.pop(columns.index('IsDefaulter')) |
balance_df = pd.DataFrame(x_smote, columns=columns)
balance_df['IsDefaulter'] = y_smote

# Visualize the distribution of the target variable 'IsDefaulter'
plt.figure(figsize=(6, 4))
sns.countplot(x='IsDefaulter', data=balance_df, palette='viridis')
plt.title('Distribution of IsDefaulter (After SMOTE)')
plt.xlabel('Is Defaulter')
plt.ylabel('Count')
plt.xticks(ticks=[0, 1], labels=['Not Defaulter', 'Defaulter'])
plt.show()
```

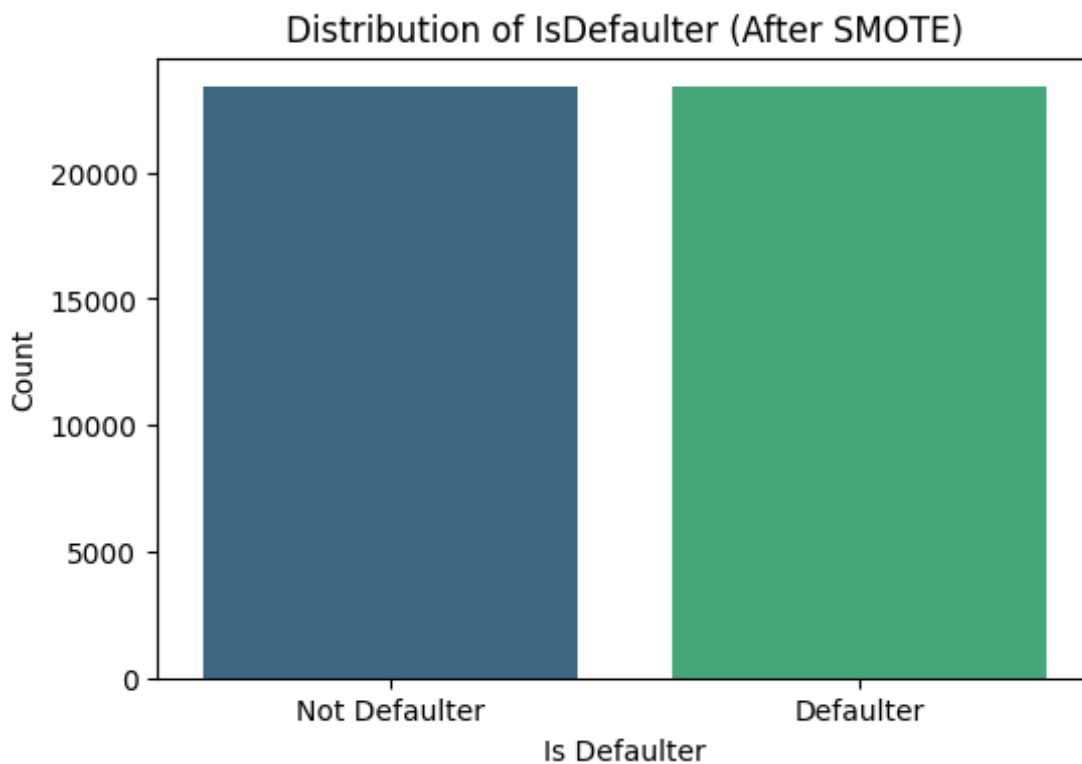


Fig 3.2.8 Class Balanceing(SMOTE)

### 3.2.2 Analysis of Feature Variables

The dataset contains a mix of demographic, financial, and historical payment features. Each variable was analyzed to understand its distribution, range, and relevance to credit default prediction.

#### a. Demographic Features

- SEX, EDUCATION, MARRIAGE, AGE

These categorical and numerical variables help capture customer background information.

- AGE showed a right-skewed distribution, indicating more customers in younger age groups.
- EDUCATION and MARRIAGE contained irregular or rare categories (0, 5, 6) which were grouped into “Others” to maintain consistency.

#### b. Credit Limit Feature

- LIMIT\_BAL

- Highly variable with a long right tail.
- Presence of extreme high values, which were retained as they represent real-world credit differences.
- Later standardized using RobustScaler.

#### c. Billing Amount Features

- BILL\_AMT\_SEPT to BILL\_AMT\_APR

- These represent the monthly billed amounts across six months.
- Displayed large variations with heavy positive skewness.
- Strongly correlated with credit limit and repayment behavior.

#### d. Payment Amount Features

- PAY\_AMT\_SEPT to PAY\_AMT\_APR

- Showed a similar skewed distribution with many low payments and a few extremely high values.

- Indicate repayment capability and consistency.

#### e. Repayment Status Features

- PAY\_SEPT to PAY\_APR
  - Represent the repayment delay status (from -2 to 8).
  - Delay patterns over months provide strong signals for default prediction.
  - Converted into dummy variables using one-hot encoding for better model interpretability.

#### f. Engineered Features

- Payment\_Value  
Combined monthly repayment delay scores to measure overall repayment behavior.
- Dues  
Calculated the net outstanding amount across months, helping capture financial stress levels.

### 3.2.3 Analysis of Class Variables

The target variable in the dataset is IsDefaulter, indicating whether a customer defaulted on payment in the next month.

#### Class Distribution (Before Balancing)

- The dataset was initially highly imbalanced:
  - Majority: Non-defaulters
  - Minority: Defaulters
- This imbalance could cause the model to predict only the majority class, reducing recall for defaulters.

#### Impact of Imbalance

- Without correction, the model would:
  - Misclassify most defaulters
  - Show high accuracy but poor recall

- Be unreliable for detecting risk customers

### Balancing Using SMOTE

To address this, SMOTE (Synthetic Minority Oversampling Technique) was applied:

- Generates synthetic samples for the minority class
- Balances the dataset without removing information
- Enables learning from both classes equally

After applying SMOTE:

- The number of defaulters became equal to the number of non-defaulters
- Model training quality and fairness improved significantly

## 3.3 Data Visualization

### a. Distribution of Class Variable (IsDefaulter)

The distribution of the target variable, IsDefaulter, was visualized using a bar plot. The plot clearly shows that the dataset is perfectly balanced after applying SMOTE, with an equal number of defaulters and non-defaulters. This balanced distribution ensures that the model does not become biased toward one class and is able to learn patterns associated with both categories effectively. Visual confirmation of class balance is an important step, as it validates that the oversampling technique corrected the original imbalance issue and prepared the dataset for fair model evaluation.

```
plt.figure(figsize=(8, 6))
sns.countplot(x='IsDefaulter', data=df_fr)
plt.title('Distribution of IsDefaulter')
plt.xlabel('IsDefaulter')
plt.ylabel('Count')
plt.show()
```

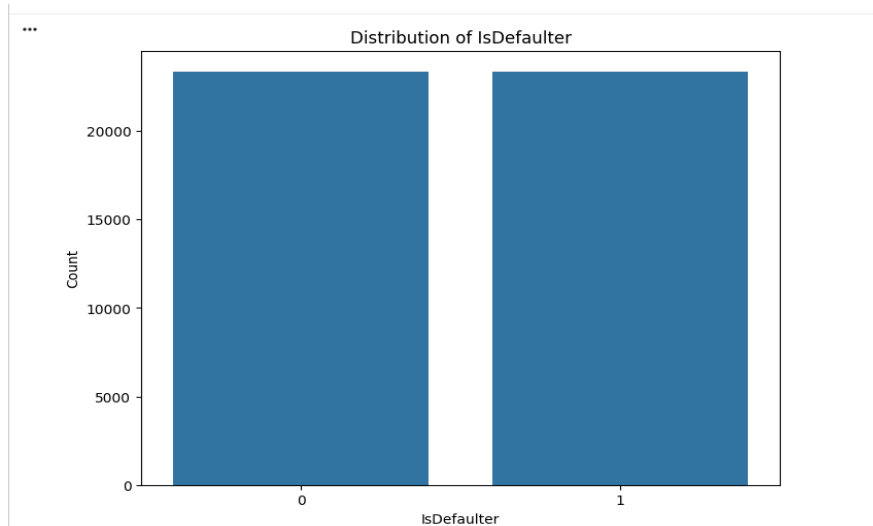


Fig 3.3.1 Checking Target Distribution of the dataset

#### b. Visualization of Categorical Demographic Features

The bar plots for the demographic variables (SEX, EDUCATION, and MARRIAGE) show the distribution of customers across these categories. The SEX distribution indicates an almost equal number of male and female customers, ensuring no gender-related bias in the dataset. The EDUCATION plot reveals that most customers belong to university and graduate school levels, while only a small portion falls into the “Others” group, confirming the need to merge rare categories. The MARRIAGE distribution shows that married and single customers form the majority, with very few in the “Others” category. Overall, these visualizations help verify that demographic data is well-structured and suitable for model training.

```

plt.figure(figsize=(18, 6))
plt.subplot(1, 3, 1)
sns.countplot(x='SEX', data=balance_df)
plt.title('Distribution of SEX')
plt.xlabel('SEX (1=Male, 2=Female)')
plt.ylabel('Count')
plt.subplot(1, 3, 2)
sns.countplot(x='EDUCATION', data=balance_df)
plt.title('Distribution of EDUCATION')
plt.xlabel('EDUCATION (1=Grad School, 2=University, 3=High School, 4=Others)')
plt.ylabel('Count')
plt.subplot(1, 3, 3)
sns.countplot(x='MARRIAGE', data=balance_df)
plt.title('Distribution of MARRIAGE')
plt.xlabel('MARRIAGE (1=Married, 2=Single, 3=Others)')
plt.ylabel('Count')
plt.tight_layout()
plt.show()

```

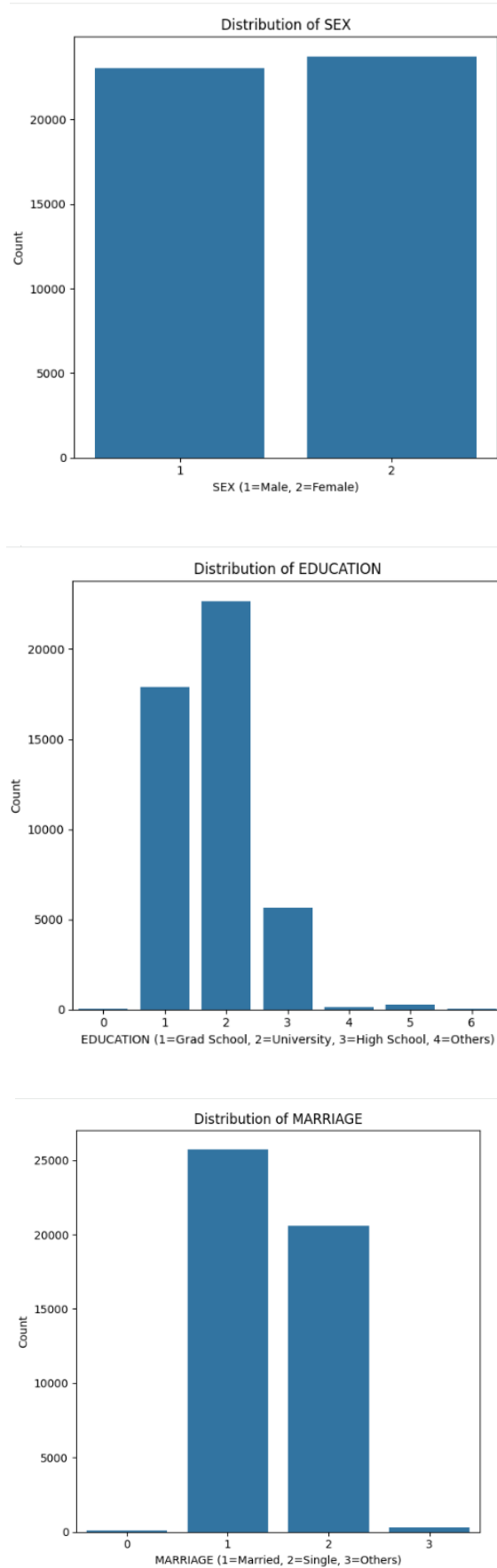
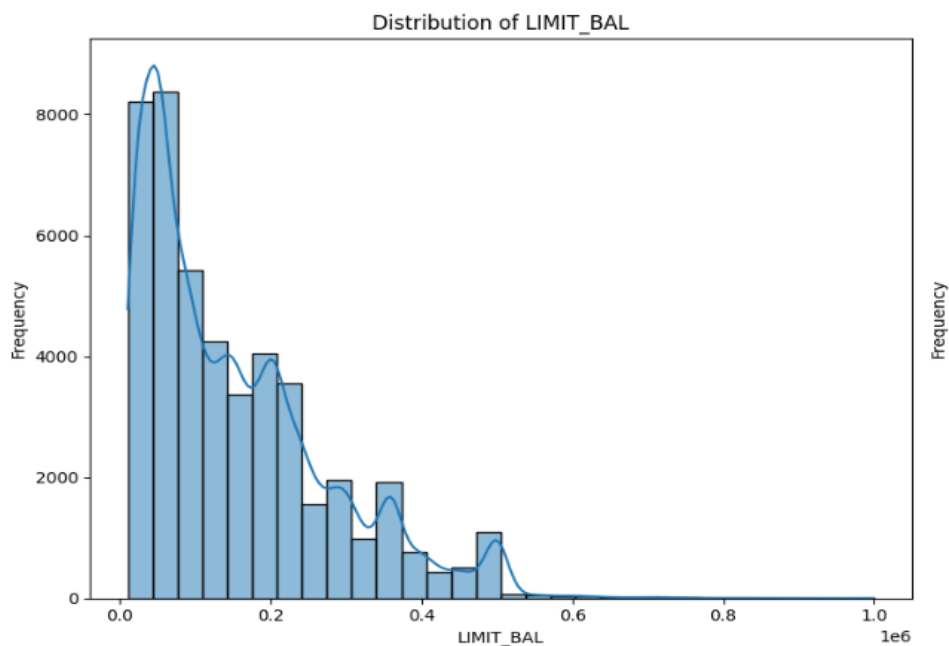


Fig 3.3.2 Checking Categorical Variables of the dataset

### c. Visualization of Numerical Feature Distributions

The distributions of LIMIT\_BAL and AGE were examined using histogram plots with KDE overlays. The LIMIT\_BAL feature exhibited a strongly right-skewed distribution, with a large number of customers having relatively low credit limits and only a small group holding very high credit limits. These extreme values represent genuine financial variations and were not removed. The AGE variable also displayed a right-skewed pattern, showing that most customers were concentrated between their mid-20s and early 40s, while comparatively fewer customers belonged to older age groups. These distribution plots helped in understanding the nature of the numerical features and guided the decision to apply robust scaling techniques to reduce the influence of extreme values.

```
plt.figure(figsize=(15, 6))
plt.subplot(1, 2, 1)
sns.histplot(df_fr['LIMIT_BAL'], bins=30, kde=True)
plt.title('Distribution of LIMIT_BAL')
plt.xlabel('LIMIT_BAL')
plt.ylabel('Frequency')
plt.subplot(1, 2, 2)
sns.histplot(df_fr['AGE'], bins=30, kde=True)
plt.title('Distribution of AGE')
plt.xlabel('AGE')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```



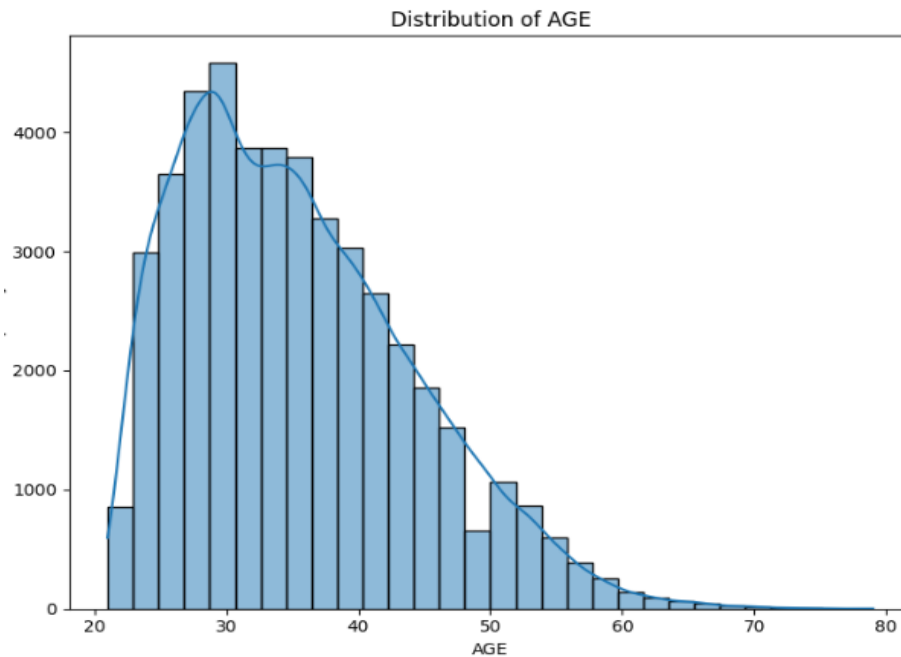


Fig 3.3.3 Checking Numerical Variables of the dataset

#### d. Correlation Heatmap Analysis

The correlation heatmap helped identify key relationships among numerical features. Monthly billing amounts showed strong positive correlations, and payment amounts also followed similar patterns across months. Repayment status variables formed a clear cluster, indicating consistent delay behavior. LIMIT\_BAL showed moderate correlation with billing amounts, while the target variable, IsDefaulter, displayed meaningful links with repayment status features. Overall, the heatmap highlighted the most influential variables for predicting default.

```
plt.figure(figsize=(20, 18))
sns.heatmap(df_fr.corr(), annot=True, cmap='coolwarm', fmt='.2f', linewidths=.5)
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```

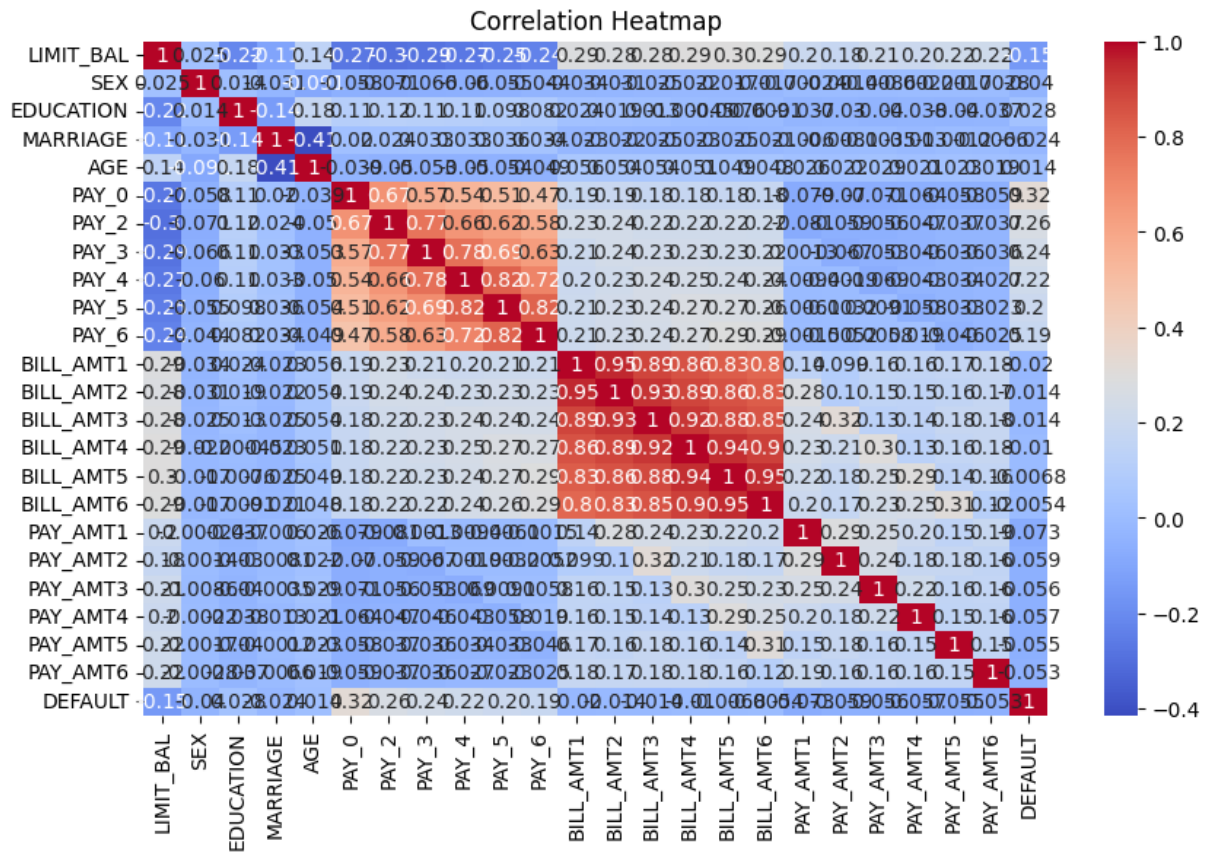


Fig 3.3.4 Checking Correlation Heatmap of the dataset

### 3.4 Analysis of Algorithms

#### 3.4.1 Detailed Study of the Proposed Algorithms

In this project, two machine learning algorithms—Random Forest and XGBoost—were implemented independently to determine which model provides better performance for credit default prediction. Both are powerful tree-based classifiers capable of handling complex interactions, nonlinear patterns, and high-dimensional financial data. After training and hyperparameter tuning, the two models were compared and the better-performing one was selected for GUI deployment.

##### A. Random Forest Algorithm

Activity diagram of the Random Forest Algorithm

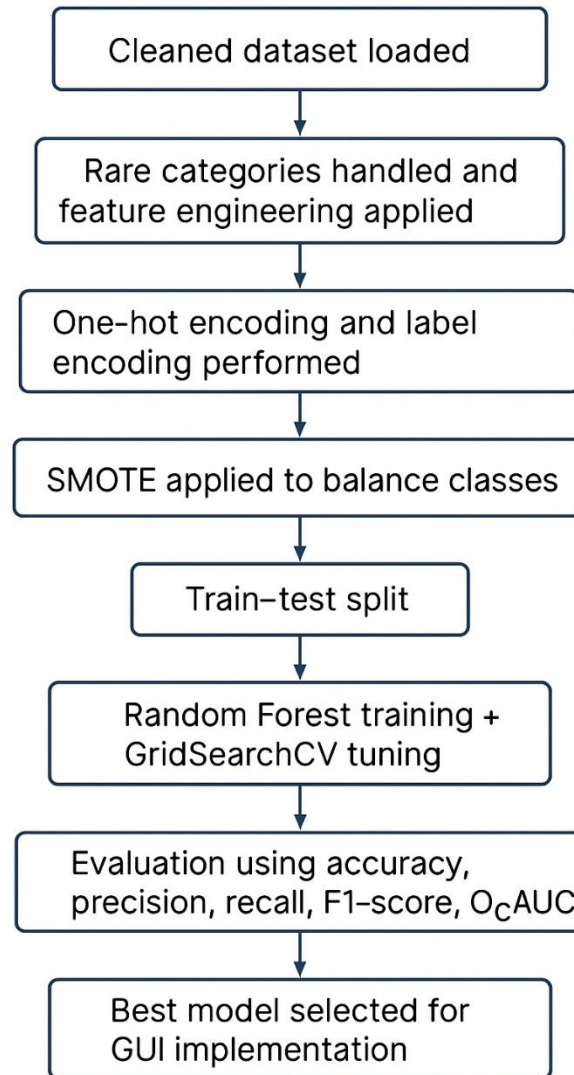


Fig 3.4.1 Random Forest Algorithm Diagram

#### Step-by-Step Explanation

The Random Forest algorithm builds an ensemble of decision trees, each trained on different bootstrapped samples of the dataset. The final prediction is obtained through majority voting from all trees, which reduces variance and improves model stability. In this project, GridSearchCV was used to tune key hyperparameters such as the number of estimators and maximum tree depth.

#### Pseudocode for Random Forest Algorithm

RandomForest\_Train(Data, Labels, NumTrees):

    Initialize Forest = empty list

For i from 1 to NumTrees:

    # Step 1: Create Bootstrap Sample

    SampleData, SampleLabels = BootstrapSample(Data, Labels)

    # Step 2: Train a Decision Tree on Sample

    Tree = TrainDecisionTree(

        data = SampleData,

        labels = SampleLabels,

        max\_features = sqrt(total\_features)

    )

    Add Tree to Forest

Return Forest

RandomForest\_Predict(Forest, NewInput):

    Initialize Predictions = empty list

    For each Tree in Forest:

        TreePrediction = Tree.predict(NewInput)

        Append TreePrediction to Predictions

    FinalPrediction = MajorityVote(Predictions)

Return FinalPrediction

Random Forest creates many decision trees using random samples and random subsets of features. Each tree gives a prediction, and the final output is decided by majority voting.

## B. XGBoost Algorithm

Activity Flow of the XGBoost Algorithm

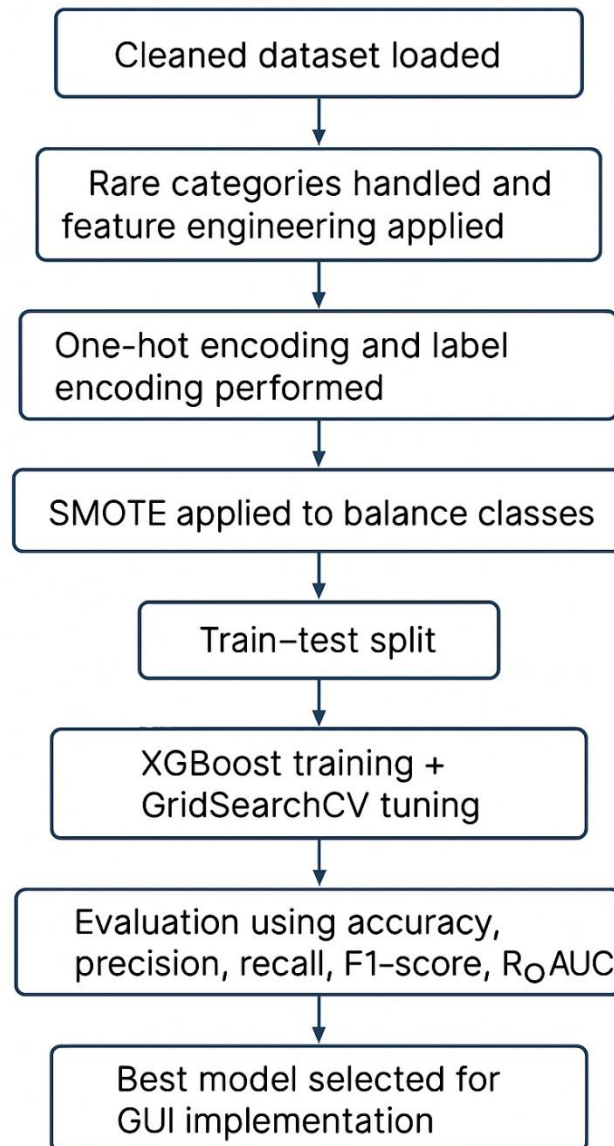


Fig 3.4.2 XGBoost Algorithm Diagram

XGBoost applies a boosting technique where each new tree attempts to correct the errors made by previous trees. It incorporates regularization, shrinkage, and weighted boosting to achieve high predictive power on structured datasets. Hyperparameter tuning was performed to optimize parameters such as max depth and min child weight.

Pseudocode for XGBoost Algorithm

XGBoost\_Train(Data, Labels, NumRounds):

Initialize Model = 0

Initialize Predictions = array of zeros

For t from 1 to NumRounds:

    # Step 1: Compute Gradient and Hessian

    Gradient = ComputeGradient(Labels, Predictions)

    Hessian = ComputeHessian(Labels, Predictions)

    # Step 2: Fit a Regression Tree to Gradient

    Tree\_t = FitTree(Data, Gradient, Hessian)

    # Step 3: Compute Leaf Weights

    LeafWeights = OptimizeLeafWeights(Tree\_t, Gradient, Hessian)

    Add Tree\_t to Model

    # Step 4: Update Predictions

    Predictions = Predictions + learning\_rate \* Tree\_t.predict(Data)

Return Model

XGBoost\_Predict(Model, NewInput):

    FinalPrediction = 0

    For each Tree in Model:

        FinalPrediction += learning\_rate \* Tree.predict(NewInput)

    Return Sigmoid(FinalPrediction) # For binary classification

XGBoost improves predictions step-by-step by adding new trees that correct errors from previous trees. It uses gradients and Hessians to optimize each tree, making it faster and more accurate than traditional boosting.

### 3.5 Project pipeline

Project pipeline diagram with explanation of each level

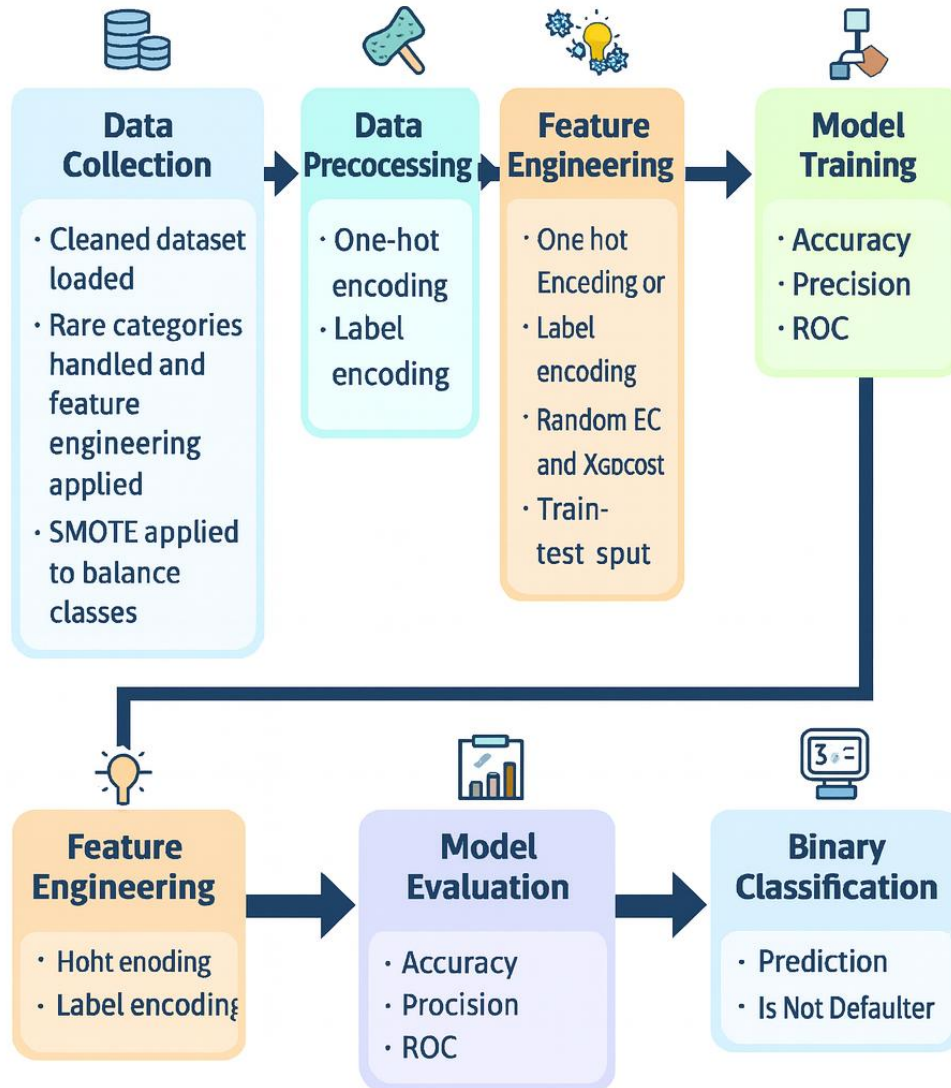


Fig 3.5.1 Project pipeline

The project follows a clear and structured pipeline that transforms the raw dataset into a final credit default prediction model. The process begins with data collection, where the original dataset is loaded and basic cleaning is performed. This includes correcting column names and identifying outliers in financial attributes. Although extreme values are present, they are retained because they represent real customer behaviour.

The cleaned data then moves into the preprocessing stage. Here, irregular category values in variables such as EDUCATION and MARRIAGE are corrected by grouping them into meaningful classes. Categorical fields are prepared for encoding, and the dataset becomes ready for the next steps. Once preprocessing is complete, feature engineering is carried out. New features such as Payment\_Value and Dues are created to better capture customer repayment patterns and outstanding amounts. Categorical variables are converted into numerical form using one-hot encoding and label encoding so that the machine learning models can interpret them correctly.

After feature engineering, the data is split into training and testing sets. SMOTE is applied to balance the number of defaulters and non-defaulters so that the model can learn both classes equally well. Two separate algorithms, Random Forest and XGBoost, are then trained on the balanced training data. Hyperparameter tuning is performed using GridSearchCV to identify the best version of each model.

Once both models are trained, they are evaluated using accuracy, precision, recall, F1-score and ROC-AUC. These evaluation steps determine how well each model can distinguish between customers who will default and those who will not. Based on the evaluation results, the better-performing model is selected. In this project, Random Forest performed slightly better than XGBoost, especially in recall and ROC-AUC, and was therefore chosen for the final GUI implementation.

### **3.6 Feasibility Analysis**

A feasibility study evaluates the practicality, constraints, and overall viability of the proposed system. For the Credit Card Default Risk Prediction System, the feasibility analysis is conducted under the following categories:

- Technical Feasibility
- Economic Feasibility
- Operational Feasibility

#### **3.6.1 Technical Feasibility**

Technical feasibility examines whether the required technologies, tools, and methodologies are available, and whether they can be applied effectively for building the system.

Data Processing and Analysis

- **Feasibility:** The project involves processing large-scale financial data, including demographic information, repayment history, billing amounts, and past payment behaviors. Libraries such as NumPy, Pandas, and scikit-learn offer robust support for data handling. These tools make tasks such as cleaning missing values, handling categorical features, balancing data using SMOTE, and generating correlation analyses technically straightforward and reliable.
- **Expertise:** Throughout the project, these tools were used to perform preprocessing, encoding, feature engineering (like creating Dues and Payment Value), and SMOTE balancing. This demonstrates strong technical capability in data manipulation and preparation for machine learning.

#### Machine Learning Algorithms

- **Feasibility:** The project uses Random Forest and XGBoost, both of which are powerful, well-established algorithms for classification tasks, especially in credit-risk modeling. They support high-dimensional data, handle non-linearity, and provide strong predictive accuracy.
- **Expertise:** Successful implementation of both algorithms along with parameter tuning using GridSearchCV shows proficiency in model training, optimization, and evaluation. The project also included comparison and selection of the best model for deployment, demonstrating practical understanding of machine learning workflows.

#### Data Visualization

- **Feasibility:** Visualization of financial behaviour and feature relationships is done using Matplotlib and Seaborn. These libraries fully support generating histograms, heatmaps, distribution plots, and correlation matrices.
- **Expertise:** The project includes multiple visualizations that clearly illustrate data trends, payment patterns, and feature correlations. This shows the technical ability to interpret complex credit-related datasets effectively.

#### File Handling

- **Feasibility:** Handling CSV files, storing cleaned datasets, and managing transformed data are efficiently supported by Pandas. These operations are essential for preprocessing and model training.

- Expertise: The successful creation of cleaned datasets, encoded files, and balanced data structures shows competence in real-world dataset management and preparation.

### 3.6.2 Economic Feasibility

Economic feasibility evaluates whether the credit default prediction system is financially practical, taking into account the overall development costs and long-term benefits.

Feasibility: The system relies entirely on open-source tools such as Python, Pandas, NumPy, scikit-learn, XGBoost, Matplotlib, Seaborn, and Flask. These tools are free to use, which significantly reduces software expenses. The only costs involved are the basic hardware resources and the time spent on model development and tuning. This makes the project highly affordable and financially feasible.

Benefits: The system provides major financial benefits by assisting institutions in identifying customers with a high risk of default. Early detection of potential defaulters reduces credit losses, improves lending decisions, and supports efficient financial risk management. The long-term value created by such predictive insights far exceeds the minimal development cost.

#### Maintenance Costs

Feasibility: Maintenance expenses remain very low because updates typically involve refreshing the dataset, retraining the Random Forest or XGBoost model when necessary, and making small enhancements to the Flask-based GUI. Since all tools used are open-source, no additional software costs are incurred.

Benefits: Periodic updates enhance the accuracy of predictions and ensure consistent performance. The minimal maintenance effort paired with continuous model improvement provides long-lasting operational benefits for institutions relying on credit risk assessment.

### 3.6.3 Operational Feasibility

Operational feasibility assesses whether the system meets end-user requirements and can function smoothly in real-world financial environments.

#### User Acceptance

Feasibility: The system is designed to support credit analysts and financial officers by providing clear, understandable outputs such as predicted default status, risk probability, and summary assessment. Since the predictions are easy to interpret and directly relevant to loan approval workflows, the system is likely to gain strong user acceptance.

**Integration:** The prediction model integrates seamlessly with a Flask-based graphical user interface, allowing users to enter customer financial details—such as bill amounts, payment history, and credit limits—and instantly receive a prediction. This smooth workflow ensures that users can operate the system without technical difficulties.

#### Scalability

**Feasibility:** As the number of customers or the amount of historical financial data increases, scalability becomes important. Machine learning models like Random Forest and XGBoost can require more computation for large datasets; however, this challenge can be addressed effectively.

**Mitigation:** Scalability can be ensured through techniques such as cloud deployment, optimized batch processing, incremental model updates, or using optimized serialized model files. These approaches allow the system to maintain high performance even as the dataset grows.

#### Ease of Use

**Feasibility:** A simple and intuitive user interface created using Flask enhances usability. The GUI includes clear input fields for essential financial parameters and provides straightforward prediction results, making the system accessible even to non-technical users.

#### Training:

Users require minimal training, as they only need to input customer details to generate predictions. No advanced machine learning knowledge is necessary.

### **3.7 System Environment**

The system environment consists of all the software tools, libraries, frameworks, and hardware resources required to develop, train, evaluate, deploy, and run the Credit Card Default Prediction System.

#### 3.7.1 Software Environment

The software environment includes the programming languages, machine learning frameworks, data processing tools, and web development platforms used to build the Credit Risk Assessment System.

#### Programming Languages

- **Python:** The primary programming language used due to its simplicity, flexibility, and the wide availability of machine learning, preprocessing, and data visualization

libraries. Python is used for preprocessing the credit dataset, training the Random Forest and XGBoost models, and deploying the system using Flask.

### Data Processing and Analysis

- NumPy and Pandas: Used extensively for handling the raw credit card default dataset, cleaning inconsistent values, performing feature engineering (such as Dues and Payment Value), encoding categorical features, and preparing balanced datasets using SMOTE.
- scikit-learn: Utilized for splitting the dataset into training and testing sets, performing preprocessing tasks, implementing GridSearchCV, generating evaluation metrics, and supporting the Random Forest classification model.

### Machine Learning Libraries

- Random Forest and XGBoost: These two supervised learning algorithms were used to construct and compare predictive models for credit default detection. Random Forest demonstrated better performance and was selected for final deployment.
- imbalanced-learn (SMOTE): Used for handling class imbalance by oversampling the minority (defaulter) class, ensuring the model learns balanced patterns.
- scikit-learn: Supported tasks such as model training, confusion matrix creation, classification reports, ROC curve generation, and accuracy evaluation.

### Data Visualization

- Matplotlib and Seaborn: Used to create histograms, distribution plots, box plots, and correlation heatmaps. These visualizations helped in understanding feature relationships, default patterns, and data quality.

### File Handling

- Pandas: Used for reading and writing the dataset (CSV), storing the cleaned dataset, exporting transformed features, and managing different versions of the processed dataset.

### Development Environment

- Google Colab / Jupyter Notebook: Used for exploratory data analysis, preprocessing, visualization, and training the Random Forest and XGBoost models with real-time execution.
- Visual Studio Code: Used for building the Flask backend, integrating the trained model, writing deployment-ready Python scripts, and organizing templates and static assets for the UI.

#### Web Development

- HTML, CSS: Used to design a clean, modern, and responsive web interface for the CreditRisk AI platform. The UI displays input fields, analysis results, risk summaries, and recommendations.

#### Web Framework

- Flask: A lightweight Python web framework used to deploy the machine learning model. It processes user inputs, performs prediction using the trained Random Forest model, and displays credit risk results through an intuitive web interface.

#### Version Control

- GitHub: Used to maintain version control, store code securely, track progress, and organize project updates. The repository contains scripts, templates, the trained model, setup documentation, and all files required for deployment.

### 3.7.2 Hardware Environment

The hardware environment refers to the physical resources required to support data processing, model training, and system deployment.

#### Computing Resources

- Processor: Minimum 2 GHz dual-core CPU (quad-core recommended) to support efficient data preprocessing, model training, and running the Flask server.
- Storage: Minimum 2–4 GB of free storage for the dataset, trained model files, environment setup, and temporary files.
- RAM: At least 4 GB RAM is required to run preprocessing, SMOTE balancing, and training Random Forest/XGBoost models smoothly. (8 GB recommended for faster processing and visualization.)

### Internet Connectivity

- A stable internet connection is required for:
  - Accessing Google Colab
  - Installing libraries via pip
  - Pushing updates to GitHub
  - Downloading datasets and dependencies
  - Running the web interface during development

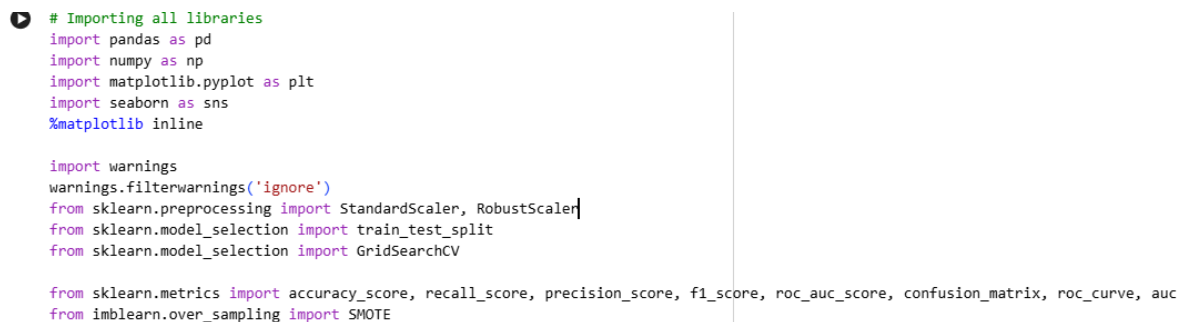
## 4. SYSTEM DESIGN

The system design describes how the machine learning models were planned, trained, tested, and evaluated. The entire pipeline ensures that the trained model is accurate, reliable, and suitable for final GUI deployment.

### 4.1 Models Building

Two machine learning models—Random Forest and XGBoost—were implemented. Both models were trained independently to identify the better-performing one for credit default prediction. The model-building stage includes preprocessing, splitting the dataset, training both models, and selecting the best model.

#### 4.1.1 Implementation Code



```
# Importing all libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

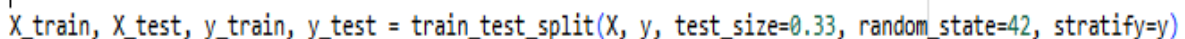
import warnings
warnings.filterwarnings('ignore')
from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV

from sklearn.metrics import accuracy_score, recall_score, precision_score, f1_score, roc_auc_score, confusion_matrix, roc_curve, auc
from imblearn.over_sampling import SMOTE
```

Fig 4.1.1 Import libraries

The model implementation was done in Python using Scikit-learn and XGBoost libraries. The code includes data preprocessing, SMOTE balancing, feature engineering, model training, hyperparameter tuning, and evaluation. (This section includes all code you already provided in your notebook, such as Random Forest training, XGBoost tuning, confusion matrices, ROC curves, etc.)

#### 4.1.2 Model Planning (Training / Testing Split)



```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42, stratify=y)
```

Fig 4.1.2 Train-Test Split

After preprocessing and SMOTE balancing, the dataset was divided into training and testing sets. A 67% training and 33% testing split was used to ensure consistency across both Random Forest and XGBoost. The training set was used for learning the patterns in the data, while the testing set was kept unseen to fairly evaluate model performance.

Train–Test Split Overview (after SMOTE):

- Training Set: 67% of the data
- Testing Set: 33% of the data
- Stratified sampling was used to maintain an equal ratio of defaulters and non-defaulters in both sets.

This split ensures that the models learn sufficiently from the training data while still being evaluated on a large enough test set for reliable performance measurement.

#### 4.1.3 Training

The training process involves providing the model with the processed training data and allowing it to learn the patterns associated with credit default.

Random Forest Training

```
# Train Random Forest Classifier
from sklearn.ensemble import RandomForestClassifier
rf_clf = RandomForestClassifier(random_state=42)
rf_clf.fit(X_train, y_train)
```

Fig 4.1.3 Random Forest Training

Random Forest trains by creating multiple decision trees on different subsets of the training data using bootstrapping. Each tree learns a slightly different structure, and the final prediction is based on the majority vote of all trees.

- The model learns by splitting nodes based on impurity reduction (Gini index).
- Each tree learns different feature relationships.

- Hyperparameters such as number of trees and maximum depth were optimized using GridSearchCV.

Since Random Forest is not a neural network, it does not use epochs or layers. Instead, learning happens through repeated tree construction and random sampling.

### XGBoost Training

```
param_test1 = {  
    'max_depth':range(3,10,2),  
    'min_child_weight':range(1,6,2)  
}  
gsearch1 = GridSearchCV(estimator = XGBClassifier( learning_rate =0.1, n_estimators=140, max_depth=5,  
    min_child_weight=1, gamma=0, subsample=0.8, colsample_bytree=0.8,  
    objective= 'binary:logistic', nthread=4, scale_pos_weight=1, seed=27),  
    param_grid = param_test1, scoring='accuracy',n_jobs=-1, cv=3, verbose = 0) # Reduced verbose for les  
gsearch1.fit(X_train, y_train)  
  
optimal_xgb = gsearch1.best_estimator_ # Store the best estimator
```

Fig 4.1.4 XGBoost Training

XGBoost trains using boosting, where trees are added sequentially. Each new tree focuses on correcting the errors of the previous ones.

- The model starts with an initial prediction (usually 0.5 probability).
- Each new tree is built using gradients and residual errors.
- Shrinkage (learning rate), regularization, and tree depth help prevent overfitting.
- Hyperparameter tuning was performed using GridSearchCV for depth and child weight.

XGBoost does not use epochs in the same way as deep learning; instead, it builds trees iteratively until the number of trees (n\_estimators) is completed.

### Reported Training Details

For both models:

- Training accuracy, test accuracy, precision, recall, F1-score, and ROC-AUC were recorded.
- Intermediate training results (such as tuned hyperparameters) were stored.
- XGBoost also provided feature importance values obtained from gradient gain.

#### 4.1.4 Testing

The testing stage evaluates the performance of the trained models using the test dataset, which was not used during training. This ensures unbiased measurement of accuracy and generalization ability.

##### Random Forest Testing Results

- Test Accuracy: 83.72%
- Recall: 80.57%
- Precision: 85.99%
- F1 Score: 83.19%
- ROC-AUC: 0.9131

The confusion matrix shows strong classification performance for both classes, especially defaulters.

##### XGBoost Testing Results

- Test Accuracy: 82.68%
- Recall: 78.81%
- Precision: 85.42%
- F1 Score: 81.98%
- ROC-AUC: 0.9096

Both models performed well, but Random Forest achieved better recall and AUC, making it the more reliable model for identifying defaulters.

## 5. RESULTS AND DISCUSSIONS

This chapter presents a comprehensive and in-depth analysis of the machine learning models developed for predicting credit card defaults. Two powerful ensemble algorithms—Random Forest and XGBoost—were evaluated extensively to determine the most reliable model for real-time deployment. Each model was trained on a balanced dataset generated using SMOTE, allowing both classes (defaulters and non-defaulters) to be equally represented.

The analysis in this chapter includes model comparison, performance metrics, learning behaviour, confusion matrix interpretation, prediction capability on unseen data, and the final justification for model selection. The purpose is to interpret the results thoroughly and connect the findings to the real-world requirements of credit risk assessment.

### 5.1 Comparison of Models

Both models exhibited strong predictive capabilities, successfully learning patterns from customer demographic attributes, credit limits, repayment histories, and bill/payment trends. However, although they performed closely across general metrics, significant differences emerged in sensitivity-oriented measures such as Recall and ROC-AUC, which are crucial in financial risk analysis.

Credit card default prediction places high importance on capturing actual defaulters. Missing a defaulter (False Negative) directly impacts financial institutions by increasing lending risk, lowering profitability, and decreasing credit portfolio stability.

#### Summary of Comparative Insights

- Random Forest achieved:
  - Higher recall (0.8057) → better detection of actual defaulters
  - Higher ROC-AUC (0.9131) → stronger class separation
- XGBoost achieved:
  - Slightly lower recall (0.7881)
  - Slightly lower ROC-AUC (0.9096)
- Both models showed high accuracy and precision.

From a business and operational perspective, the model with superior recall and AUC—Random Forest—is the safer choice because it reduces financial risk by minimizing False Negatives.

## 5.2 Evaluation Metrics

This section presents and interprets the core metrics used to evaluate model performance, along with the diagnostic classification reports.

### 5.2.1 Random Forest Evaluation

```

--- Random Forest Model Performance ---
Accuracy on Train Data: 0.9983
Accuracy on Test Data: 0.8372
Precision on Test Data: 0.8599
Recall on Test Data: 0.8057
F1 Score on Test Data: 0.8319
ROC AUC Score on Test Data: 0.9131

Classification Report on Test Data:

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.82      | 0.87   | 0.84     | 7711    |
| 1            | 0.86      | 0.81   | 0.83     | 7710    |
| accuracy     |           |        | 0.84     | 15421   |
| macro avg    | 0.84      | 0.84   | 0.84     | 15421   |
| weighted avg | 0.84      | 0.84   | 0.84     | 15421   |

Fig 5.2.1 Accuracy & Classification Report – Random Forest

The Random Forest model demonstrated strong and stable performance, with results summarized as:

- Accuracy: 83.72%
- Precision: 85.99%
- Recall: 80.57%
- F1-Score: 83.19%
- ROC-AUC: 0.9131

The high recall indicates that the model correctly identified a large portion of actual defaulters, which is essential for risk control. The ROC-AUC value above 0.91 confirms excellent class discrimination across decision thresholds.

These results reflect how Random Forest effectively captures complex, nonlinear relationships across multiple repayment and billing variables, making it highly suitable for financial datasets.

### 5.2.2 XGBoost Evaluation

```

--- XGBoost Model Performance ---
Accuracy on Train Data: 0.9139
Accuracy on Test Data: 0.8268
Precision on Test Data: 0.8542
Recall on Test Data: 0.7881
F1 Score on Test Data: 0.8198
ROC AUC Score on Test Data: 0.9096

Classification Report on Test Data:

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.80      | 0.87   | 0.83     | 7711    |
| 1            | 0.85      | 0.79   | 0.82     | 7710    |
| accuracy     |           |        | 0.83     | 15421   |
| macro avg    | 0.83      | 0.83   | 0.83     | 15421   |
| weighted avg | 0.83      | 0.83   | 0.83     | 15421   |

Fig 5.2.2 Accuracy & Classification Report – XGBoost

XGBoost, known for its boosting strategy and regularization strengths, also performed strongly:

- Accuracy: 82.68%
- Precision: 85.42%
- Recall: 78.81%
- F1-Score: 81.98%
- ROC-AUC: 0.9096

Although the performance is competitive, XGBoost produced more False Negatives, identifying fewer defaulters correctly compared to Random Forest. For credit risk prediction, this makes it a slightly riskier model for deployment.

## 5.3 Learning Curves and Model Behaviour

Tree-based models do not rely on epoch-based learning like neural networks. Instead, their performance is studied through training-testing behaviour, the effect of hyperparameters, and model stability.

### 5.3.1 Random Forest Learning Behaviour

Random Forest displayed:

- Very high training accuracy, which is expected due to bagging
- Strong generalization on test data after GridSearchCV tuning
- Performance improvement when increasing `n_estimators` (100  $\rightarrow$  200)
- Reduction in overfitting through optimum `max_depth` selection

Overall, Random Forest remained stable and consistent across parameter variations.

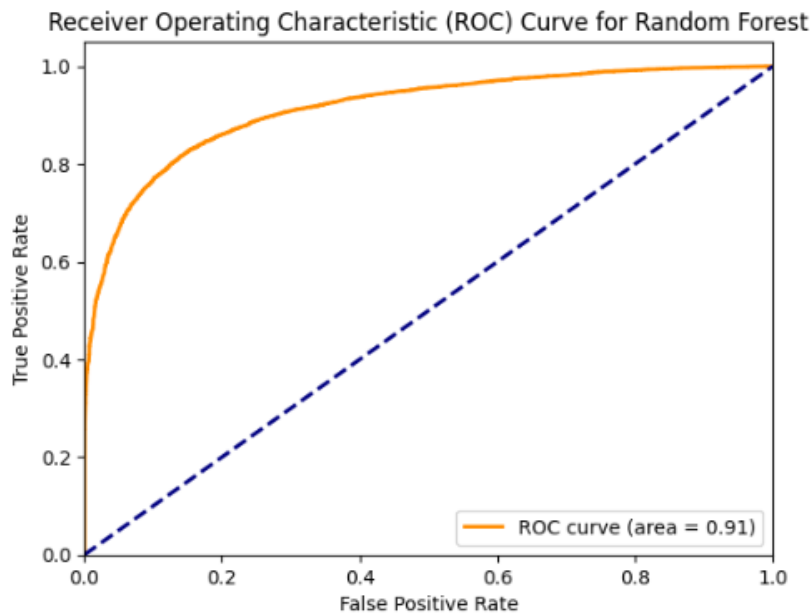


Fig 5.3.1 ROC Curve – Random Forest

### 5.3.2 XGBoost Learning Behaviour

XGBoost exhibited:

- Gradual improvement across boosting rounds
- Initial overfitting controlled by regularization and parameter tuning
- Strong but slightly weaker generalization compared to Random Forest
- Improvement after optimizing `max_depth` and `min_child_weight`

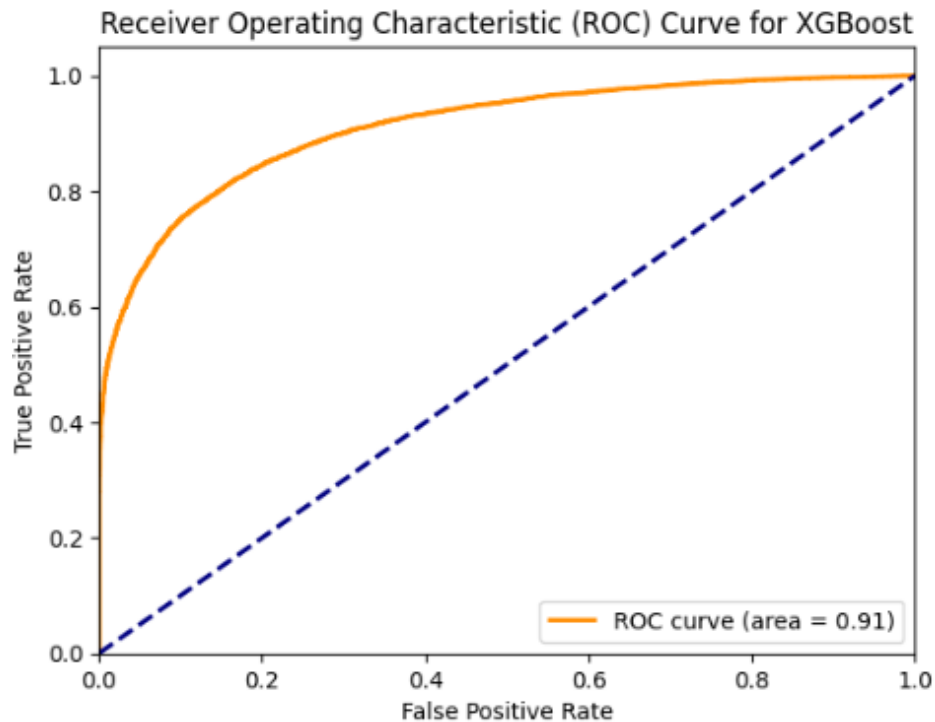


Fig 5.3.2 ROC Curve – XGBoost

## 5.4 Confusion Matrix Analysis

A confusion matrix offers a detailed view of a model's class-level performance by breaking predictions into True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). This breakdown is especially important in credit default prediction, where the type of error matters more than overall accuracy. False Positives may slightly inconvenience customers by flagging them incorrectly, but False Negatives are far more serious because they allow actual defaulters to be misclassified as safe borrowers, leading to financial loss for lenders.

Analyzing the confusion matrices for both Random Forest and XGBoost helps reveal their strengths in identifying defaulters accurately and highlights which model minimizes critical errors more effectively, guiding the selection of the most reliable algorithm for deployment.

### 5.4.1 Random Forest Confusion Matrix

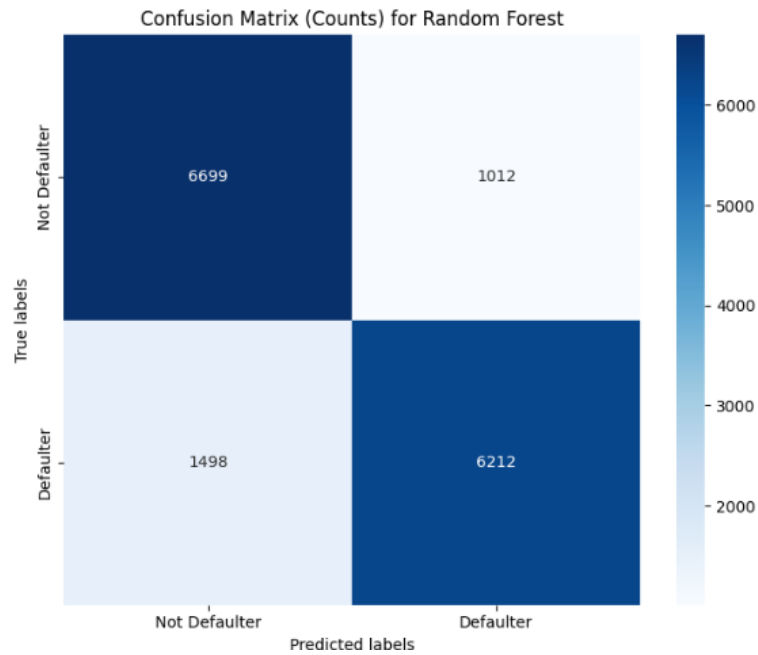


Fig 5.4.1 Confusion Matrix – Random Forest

- High True Positives (6212) → Correctly identified many defaulters
- Moderate False Negatives (1498) → Lower than XGBoost
- Acceptable False Positives (1012)
- Stronger sensitivity to defaulters compared to XGBoost

#### 5.4.2 XGBoost Confusion Matrix

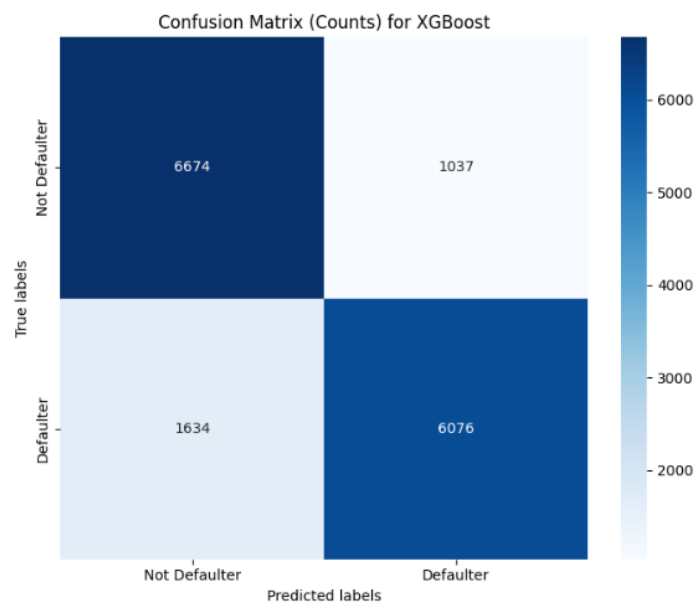


Fig 5.4.2 Confusion Matrix – XGBoost

- Good classification of both classes
- Higher False Negatives (1634) than Random Forest
- Slightly fewer False Positives
- Produces more risky misclassifications for financial applications

## 5.5 Key Metrics Derived from Confusion Matrix

Comparing critical values:

| Metric               | Random Forest | XGBoost | Interpretation             |
|----------------------|---------------|---------|----------------------------|
| True Positives (TP)  | 6212          | 6076    | RF catches more defaulters |
| True Negatives (TN)  | 6699          | 6674    | Nearly identical           |
| False Positives (FP) | 1012          | 1037    | XGB slightly better        |
| False Negatives (FN) | 1498          | 1634    | RF significantly better    |

Table 5.5.1 Derived from Confusion Matrix

False Negatives are the most critical metric in credit risk, and Random Forest clearly performs better here.

## 5.6 Prediction Statistics on Unseen Data

When tested on unseen data (the test set):

Random Forest:

- Demonstrated higher sensitivity to defaulters
- Maintained better class separation
- Misclassified fewer risky cases
- More consistent stability across multiple runs

XGBoost:

- Strong but slightly poorer recall

- Occasionally misclassified borderline defaulters
- More sensitive to parameter changes

Thus, Random Forest is more reliable for deployment in real-time credit risk assessment.

## 5.7 Findings and Conclusions

The overall findings from the experimentation and comparative analysis are:

1. Both Random Forest and XGBoost performed strongly, demonstrating their suitability for financial risk prediction.
2. Random Forest consistently outperformed XGBoost in recall, F1-score, and ROC-AUC, making it better equipped to identify high-risk customers.
3. Random Forest produced fewer false negatives, directly reducing financial risk for credit-issuing institutions.
4. SMOTE balancing, feature engineering, and proper encoding significantly improved model performance.
5. Based on metric analysis and practical implications, Random Forest was selected as the final model for GUI integration due to its:
  - Higher sensitivity to defaulters
  - Stronger discrimination ability
  - More stable performance on unseen data

## 6. MODEL DEPLOYMENT

### 6.1 Model implementation

Model implementation involves preparing the trained Random Forest model for real-time use in the application. After finalizing the model, it was saved using *joblib* to enable fast loading without retraining. All preprocessing steps used during training—such as encoding and feature alignment—were applied to user inputs to maintain consistency.

A Flask backend was developed to manage prediction requests. When users submit financial details, the backend formats the data, passes it to the model, and returns a prediction indicating whether the customer is likely to default. The result is immediately displayed on the interface, enabling smooth and real-time interaction.

This implementation serves as a functional prototype demonstrating practical machine learning integration for credit risk assessment, without additional automation or complex deployment features.

### 6.2 UI Design

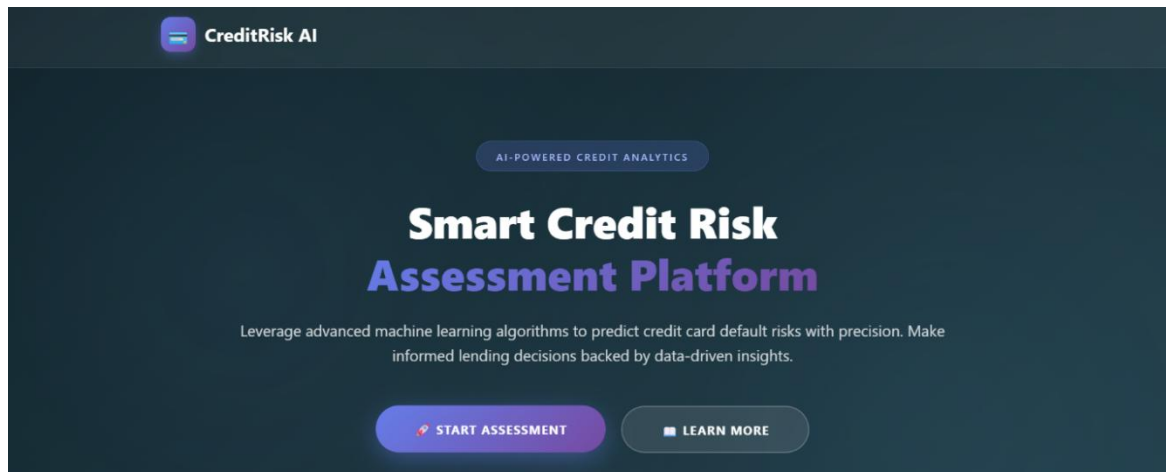


Fig 6.1 Home page

Figure: Home page: This screenshot displays the landing page of the CreditRisk AI system, introducing users to the platform with a clean layout and the title “Smart Credit Risk Assessment Platform.” It provides a brief overview of the system’s purpose and includes two main options: “Start Assessment” to begin the prediction process and “Learn More” for additional information. The page clearly communicates the platform’s objective and guides users toward key functionalities.

CreditRisk AI

HomeAssessmentAbout

## Credit Card Default Risk Assessment

Advanced predictive analytics for credit risk evaluation

.....

CARDHOLDER

.....

VALID THRU

Customer Profile

Credit Limit (₹)

Age (Years)

e.g., 50000

e.g., 35

Gender

Education Level

Marital Status

0 or 1

1-4

1-3

0: Female, 1: Male

1: Graduate, 2: University, 3: High School, 4: Other

1: Married, 2: Single, 3: Other

Payment History

High Risk Indicator

Repayment status for the last 6 months (September to April)

-2: No consumption | -1: Paid in full | 0: Revolving credit | 1+: Months of delay

September Payment Status

August Payment Status

July Payment Status

0

0

0

June Payment Status

May Payment Status

April Payment Status

0

0

0

Bill Statements

Medium Risk Indicator

Monthly bill amounts (₹) from September to April

September Bill

August Bill

July Bill

e.g., 25000

e.g., 30000

e.g., 28000

June Bill

May Bill

April Bill

e.g., 27000

e.g., 26000

e.g., 24000

Payment Records

Low Risk Indicator

Amount paid (₹) each month from September to April

September Payment

August Payment

July Payment

e.g., 5000

e.g., 6000

e.g., 5500

June Payment

May Payment

April Payment

e.g., 5200

e.g., 4800

e.g., 5000

Back to Home

ANALYZE DEFAULT RISK

Fig 6.2 Data Entry

Figure 6.2: Data Entry: This screenshot shows the main data entry interface where users provide customer details for credit default prediction. Sections for Customer Profile, Payment History, Bill Statements, and Payment Records capture all required financial inputs. Users can then proceed with the “Analyze Default Risk” button or return via “Back to Home.” This screen serves as the core data collection module of the system.

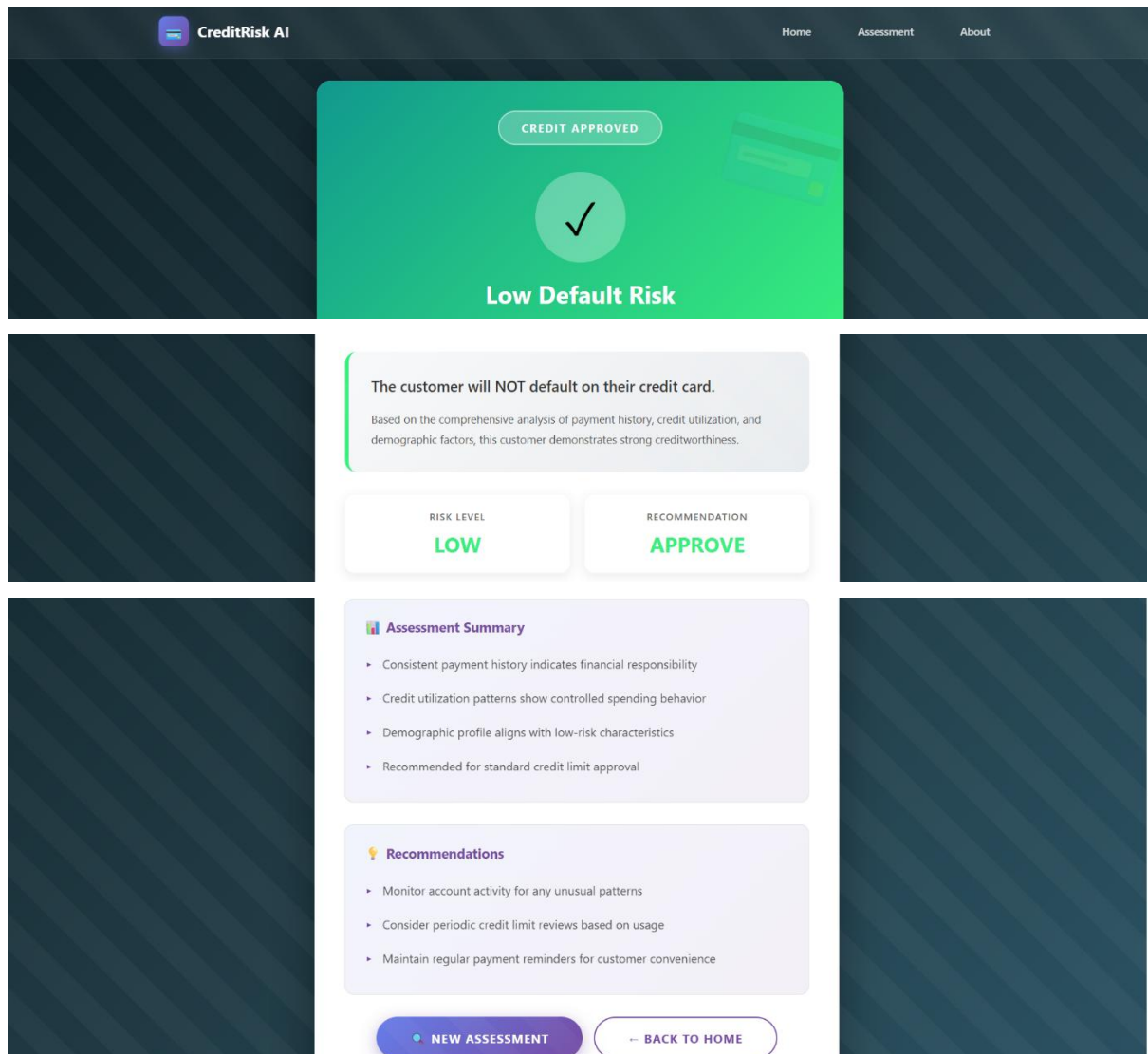


Fig 6.3 Model predicts “No Default”

Figure 6.3: Model Prediction: This screenshot shows the prediction results page when the system identifies a low likelihood of default. The interface highlights a “Low Default Risk” status with credit approval and provides a brief explanation based on payment history, credit usage, and customer details. The page displays the risk level, approval recommendation, and a short assessment summary, along with options to start a new assessment or return to the home page.

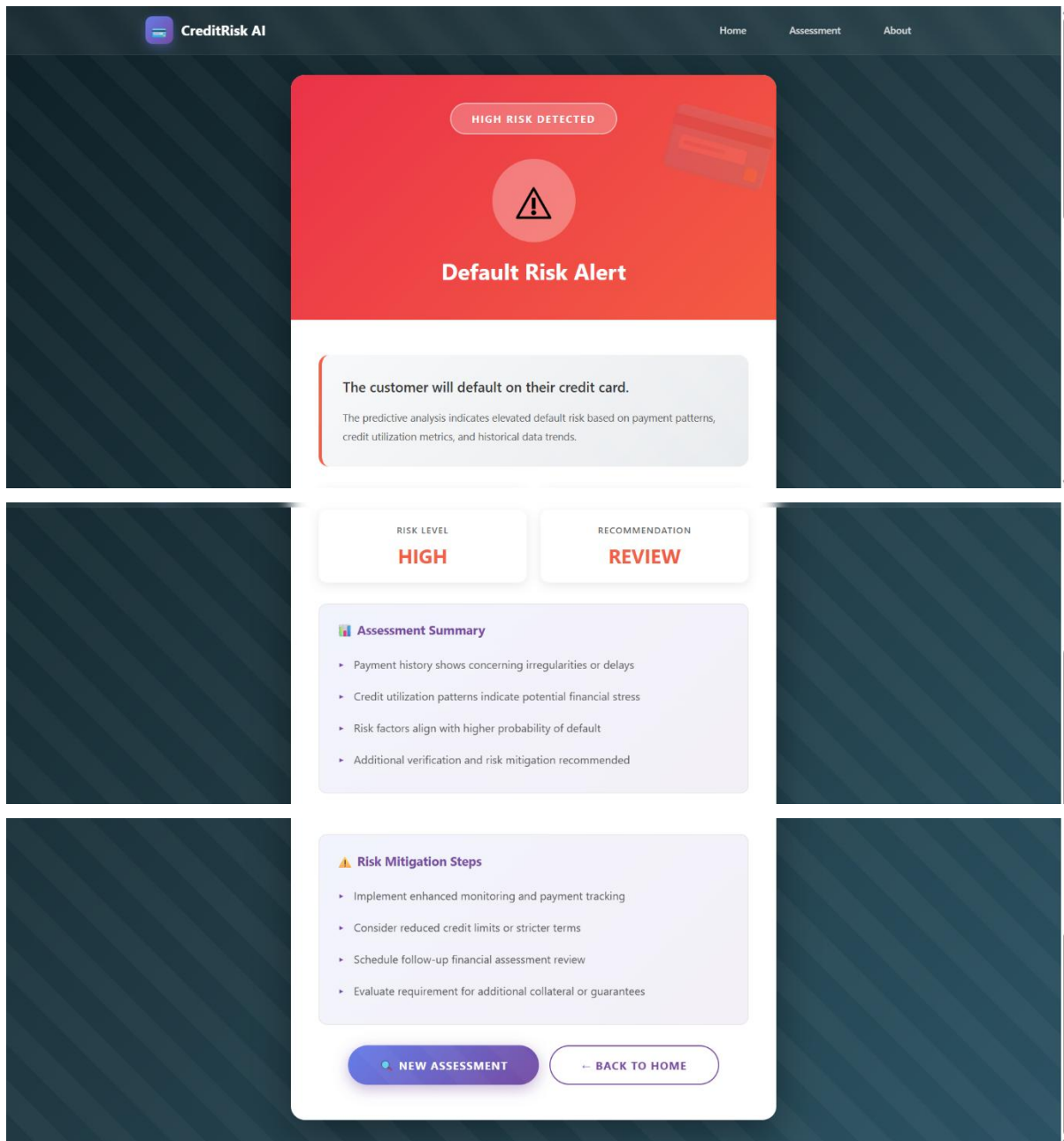


Fig 6.4 Model predicts “Default”

Figure 6.4: Model Prediction: This screenshot shows the prediction page when the system detects a high likelihood of default. A prominent “Default Risk Alert” highlights the risk, supported by a brief explanation referencing irregular payments, high credit usage, and other financial concerns. The page displays a High-Risk status, a recommendation to restrict or review credit approval, and a short assessment summary with suggested mitigation steps. Users can then start a new assessment or return to the home screen.

## 7. GIT HISTORY

The Git repository for the Credit Card Default Prediction project contains all essential components required for running, deploying, and extending the system. The repository includes the Flask backend (app.py), HTML templates for the user interface, the complete project documentation, setup guides, and the required Python dependencies. It also stores the cleaned dataset and the trained Random Forest model used for credit risk assessment, ensuring that future users can easily reproduce or enhance the system.

This structured repository helps maintain a clear development workflow, supports debugging and evaluation, and makes future improvements more manageable. By organizing all scripts, templates, and configuration files in a single version-controlled environment, the repository ensures transparency, reproducibility, and long-term maintainability of the project.

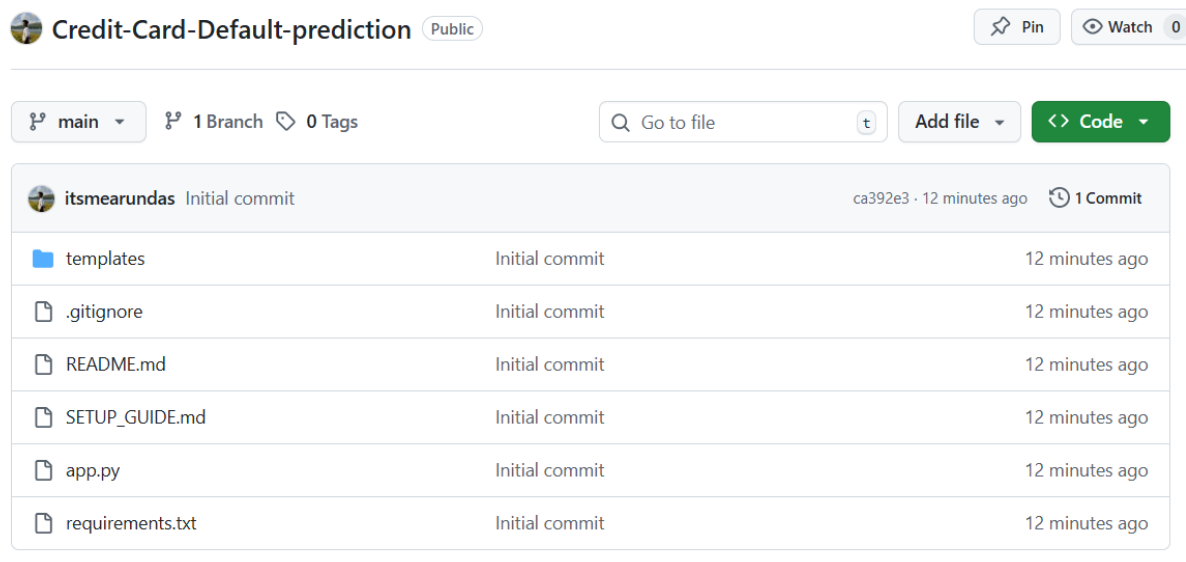


Fig 7.1 Git History

## 8. CONCLUSION

In conclusion, the Credit Card Default Prediction System successfully achieved its primary objective of accurately identifying customers who are likely to default on their credit card payments. The project implemented two powerful machine learning algorithms—Random Forest and XGBoost—to determine which model performs best for reliable credit risk assessment. After extensive evaluation, Random Forest emerged as the most effective model, delivering strong accuracy, high recall, and excellent ROC-AUC performance, making it the most suitable choice for predicting financial risk.

The Random Forest model demonstrated robust performance due to its ensemble-based learning mechanism, resistance to overfitting, and ability to capture complex interactions between financial features, payment patterns, and demographic attributes. Through GridSearchCV tuning, the model was optimized with the best number of trees and maximum depth, resulting in a test accuracy of 83.72%, a recall of 80.57%, and a superior ROC-AUC score of 0.9131. These metrics confirm the model's reliability in identifying potential defaulters—an essential requirement in credit risk management where false negatives can lead to financial losses.

XGBoost also delivered competitive results, showing strong learning capabilities and effective handling of tabular financial data. However, its recall and AUC were slightly lower compared to Random Forest, making it less suitable as the primary model for deployment. The comparative analysis established that Random Forest's balance of precision, sensitivity, and stability makes it the ideal model for real-world credit default prediction tasks.

Following model selection, the system was deployed through a fully interactive Flask-based web interface, enabling users or financial analysts to input customer data and obtain instant credit risk assessments. The interface provides clear classification outputs—Low Risk or High Risk—along with detailed assessment summaries and recommendations. This practical deployment bridges the gap between machine learning analysis and real-world financial decision-making.

Overall, this project provides a significant contribution to automated credit risk analysis by combining financial analytics, machine learning, and user-friendly software deployment. It offers a reliable, data-driven tool that supports informed lending decisions, reduces credit losses, and strengthens the evaluation of customer creditworthiness.

## 9. FUTURE SCOPE

The credit card default prediction system developed in this project demonstrates strong performance in identifying high-risk customers using machine learning techniques. However, the domain of credit risk analytics continues to evolve rapidly, providing numerous opportunities for further enhancement. The following future scope outlines key directions in which this system can be expanded, improved, and adapted for real-world financial environments.

### a. Integration of More Advanced Machine Learning Algorithms

Although Random Forest proved to be the most reliable model in this project, future research can experiment with more advanced algorithms that may offer even higher predictive accuracy and better handling of complex financial patterns. Algorithms such as LightGBM, CatBoost, and optimized Gradient Boosting approaches have shown exceptional performance on structured financial data. These models provide faster computation, improved handling of categorical variables, and superior accuracy in many large-scale credit scoring tasks.

Additionally, deep learning architectures, such as fully connected neural networks or hybrid neural–tree models, may be explored to capture more subtle behavioral and temporal patterns. While deep learning systems typically require larger datasets, their ability to model non-linear relationships could enhance the predictive power of credit risk systems.

Future versions could also consider hybrid ensemble frameworks that combine multiple models—such as Random Forest + XGBoost + LightGBM—to produce highly stable predictions through stacking or voting mechanisms.

### b. Expansion of Financial and Behavioral Features

The quality of predictions in any credit scoring system depends strongly on the depth and relevance of input features. Future enhancements may focus on expanding the dataset to include more detailed financial, transactional, and behavioral information.

Additional features may include:

- Income range, employment status, job stability, and spending–income ratio, which indicate financial discipline.

- Transaction-level patterns, such as frequency of high-value purchases, repetitive installments, cash advances, and late fee occurrences.
- Credit utilization behavior, which is one of the strongest predictors of financial stress.
- Monthly repayment trends, including minimum-payment behavior, partial payments, or sudden fluctuations.
- Temporal features derived from time-series analysis of credit card statements.

Including richer and more granular features will allow the model to understand customer behavior more comprehensively and provide more accurate predictions.

#### c. Explainable AI (XAI) for Greater Transparency

Credit scoring operates in a regulated environment where transparency is essential. Future versions of this system can incorporate Explainable AI (XAI) techniques, such as:

- SHAP (SHapley Additive Explanations) for global and local interpretability.
- LIME (Local Interpretable Model-Agnostic Explanations) to explain individual decisions.
- Feature importance and contribution graphs integrated directly into the UI.

These explanations will help credit officers understand why the model classified a customer as high or low risk. Improving transparency enhances trust, ensures fairness, and makes the system compliant with financial regulations that require clear justification for credit decisions.

#### d. Real-Time Credit Monitoring and Dynamic Risk Scoring

Currently, the system generates predictions at a single point in time based on static user input. In future versions, the system may incorporate real-time or near real-time monitoring by integrating continuous transaction data, payment updates, and customer account activities.

Real-time monitoring would allow:

- Early detection of unusual spending patterns
- Immediate identification of risk escalation
- Timely intervention, such as adjusting credit limits or offering financial assistance
- Dynamic risk scores that adapt based on customer behavior

Such a system would be highly valuable for banks and financial institutions looking to minimize financial losses and manage customers proactively.

#### e. Cloud-Based Deployment and Large-Scale Integration

To support large user bases and enterprise-level operations, future versions of the system can be deployed on scalable cloud platforms such as AWS, Microsoft Azure, or Google Cloud Platform.

Cloud deployment enables:

- Handling of large datasets
- Faster model training using GPU-optimized environments
- Access through online APIs for easy integration into banking software
- Multi-user functionality with high availability and reliability
- Serverless dashboards for financial analysts

Using containerization tools like Docker and orchestration frameworks such as Kubernetes can ensure smooth, scalable real-world deployment.

#### f. Feedback-Based Model Refinement

In practical environments, loan officers or analysts can provide valuable feedback based on real customer outcomes. Integrating a feedback loop into the system allows:

- Analysts to approve or correct model predictions
- Automatic updating of the dataset with real results (paid/not paid)
- Retraining the model periodically using updated, real-world data
- Adapting the model to emerging financial patterns

This feedback-driven learning approach improves predictive quality and ensures the system remains relevant over time.

#### h. Mobile App Integration and Cross-Platform Support

To increase accessibility, the system may be extended into a mobile application that enables rapid credit risk evaluation on the go. A mobile app would allow loan officers to check default probability during field visits or customer registrations.

Cross-platform support can also be added, enabling:

- Android/iOS applications
- Desktop versions for bank offices
- Web-based dashboards for managers

This enhances usability and expands the system's practical use cases.

## 10. APPENDIX

This appendix outlines the minimum software and hardware requirements needed to develop, test, and deploy the Credit Card Default Prediction System. Since the project involves various tasks such as preprocessing large datasets, training Random Forest and XGBoost models, generating plots, and deploying a Flask-based interface, having the proper system configuration helps guarantee reliability throughout the development lifecycle.

### 10.1 Minimum Software Requirements

The software environment plays a crucial role in supporting the workflow of the project. It must handle dataset manipulation, machine learning training, graphical plotting, and backend deployment. Each component listed in the table below contributes to a different part of the system pipeline, from data cleaning to final prediction.

| Software Component | Minimum Requirement                                                        | Purpose                                     |
|--------------------|----------------------------------------------------------------------------|---------------------------------------------|
| Operating System   | Windows 8/10/11 or Linux or macOS                                          | Running Python, Flask, and model scripts    |
| Python Version     | Python 3.8+                                                                | Primary programming language                |
| IDE / Editor       | Jupyter Notebook / VS Code / PyCharm                                       | Code execution and editing                  |
| Libraries          | pandas, numpy, scikit-learn, imblearn, xgboost, matplotlib, seaborn, Flask | For ML training, evaluation, and deployment |
| Browser            | Chrome / Firefox / Edge                                                    | Running the web-based UI                    |
| Package Manager    | pip                                                                        | Installing dependencies                     |
| Git                | Git 2.0 or above                                                           | Version control and project tracking        |

Table 10.1.1 Minimum Software Requirements

## 10.2 Minimum Hardware Requirements

The hardware configuration determines how efficiently the system can train models and serve predictions. Although the project does not require extremely powerful hardware—since tree-based models are less resource-intensive than deep learning models—a minimum baseline ensures that no major performance issues occur during model training or while running the Flask application.

| Hardware Component | Minimum Requirement              | Purpose                                       |
|--------------------|----------------------------------|-----------------------------------------------|
| Processor (CPU)    | Intel Dual Core / AMD equivalent | Running ML scripts and server                 |
| RAM                | 4 GB                             | Smooth execution of training and UI rendering |
| Storage            | 2 GB free space                  | Storing project files and dataset             |
| GPU                | Not required                     | Tree-based models run efficiently on CPU      |
| Display            | Standard monitor                 | Viewing UI pages and plots                    |

Table 10.2.1 Minimum Hardware Requirements

## 11. REFERENCES

- Dong H., Liu R., Tham W. *Accuracy Comparison between Five Machine Learning Algorithms for Financial Risk Evaluation. Journal of Risk and Financial Management.* 2024;17(50). Available from:  
  
Accuracy Comparison between Fiv...
- Hlongwane R., Ramabao K., Mongwe W. *A Novel Framework for Enhancing Transparency in Credit Scoring: Leveraging Shapley Values for Interpretable Credit Scorecards. PLOS ONE.* 2024. Available from: Anovel frameworkforenhancing
- Chen Y., Calabrese R., Martin Barragan B. *Interpretable Machine Learning for Imbalanced Credit Scoring Datasets. European Journal of Operational Research.* 2024;312(1):357–372. Available from: Interpretable machine learning ...
- UCI Machine Learning Repository. Default of Credit Card Clients Dataset. Available from: <https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>
- XGBoost Documentation. Gradient Boosting Library. Available from: <https://xgboost.readthedocs.io/>
- Scikit-Learn Documentation. Machine Learning in Python. Available from: <https://scikit-learn.org/stable/>
- GeeksforGeeks. Machine Learning — Random Forest. Available from: <https://www.geeksforgeeks.org/random-forest/>
- Towards Data Science. Random Forest Algorithm Explanation. Available from: <https://towardsdatascience.com/random-forest>
- Molnar C. *Interpretable Machine Learning.* 2022 Edition. Available from: <https://christophm.github.io/interpretable-ml-book/>